OPERATING SYSTEMS NOTES

Theory explained step by step

Author: SamuMGH04

WELCOME

Thanks for choosing these notes. This project is created with the aim of making people understand what's behind an Operating System and everything related to it.

LICENSE

These works are protected and distributed under the license
Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International

- **Attribution:** You must mention the original creator
- **Non Commercial:** You can't sell this material
- **No Derivatives:** You can't modify or change the text.

SISTEMI OPERATIVI: LEZIONE 2

Indice

1 Processi nei Sistemi Operativi Moderni **10**
1.1 Introduzione generale . . . 10
1.2 Sistemi iniziali: esecuzione di un solo programma alla volta . . . 10
1.3 Sistemi moderni: esecuzione concorrente di più programmi . . . 11
1.4 Il concetto di processo . . . 11
1.5 Unità fondamentale di lavoro nei sistemi time-sharing . . . 12
1.6 Composizione di un sistema: processi del sistema operativo e processi utente 13
1.6.1 Processi del sistema operativo . . . 13
1.6.2 Processi utente . . . 13
1.7 Idea fondamentale: concorrenza, controllo e isolamento . . . 14

2 Struttura della Memoria di un Processo **15**
2.1 Introduzione alla memoria di un processo . . . 15
2.2 Sezioni della memoria . . . 16
2.2.1 Sezione Text . . . 16
2.2.2 Sezione Data . . . 16
2.2.3 Heap . . . 17
2.2.4 Stack . . . 17
2.3 Comportamento durante l'esecuzione . . . 18
2.3.1 Chiamata di funzione . . . 18
2.3.2 Ritorno da una funzione . . . 19
2.3.3 Espansione e contrazione dell'heap . . . 19
2.4 Osservazione finale . . . 19

3 Programma e Processo **20**
3.1 Introduzione . . . 20
3.2 Il programma . . . 20
3.3 Il processo . . . 21
3.4 Quando un programma diventa un processo . . . 21
3.5 Più processi derivati dallo stesso programma . . . 22
3.5.1 Condivisione della sezione di testo . . . 22
3.5.2 Separazione delle altre sezioni di memoria . . . 22
3.6 Esempio: più istanze di un browser . . . 23
3.7 Osservazione conclusiva . . . 23

4 Stati di un Processo **24**
4.1 Introduzione al concetto di stato di un processo . . . 24
4.2 Nuovo (New) . . . 24
4.3 In esecuzione (Running) . . . 25
4.4 In attesa (Waiting) . . . 25
4.5 Pronto (Ready) . . . 26
4.6 Terminato (Terminated) . . . 26
4.7 Osservazioni sui nomi degli stati . . . 27
4.8 Numero di processi nei diversi stati . . . 27
4.9 Osservazione conclusiva . . . 27

5 Blocco di Controllo del Processo (Process Control Block) **28**
5.1 Introduzione . . . 28
5.2 Informazioni contenute nel PCB . . . 28
5.2.1 Stato del processo . . . 28
5.2.2 Program Counter . . . 29
5.2.3 Registri della CPU . . . 29
5.2.4 Informazioni di scheduling della CPU . . . 29
5.2.5 Informazioni di gestione della memoria . . . 30
5.2.6 Informazioni di contabilizzazione . . . 30
5.2.7 Stato delle operazioni di Input/Output . . . 31
5.3 Scopo del Process Control Block . . . 31
5.4 Osservazione conclusiva . . . 32

6 Dai Processi ai Thread **32**
6.1 Introduzione . . . 32
6.2 Modello tradizionale: un processo, un solo percorso di esecuzione . . . 32
6.3 Limitazioni del modello tradizionale . . . 33
6.4 Supporto ai thread nei sistemi operativi moderni . . . 33
6.5 Esecuzione simultanea di più attività . . . 34
6.6 Esecuzione parallela nei sistemi multicore . . . 34
6.7 Estensione del PCB per supportare i thread . . . 35
6.8 Osservazione conclusiva . . . 35

7 Multiprogrammazione e Time-Sharing **36**
7.1 Introduzione . . . 36
7.2 Multiprogrammazione . . . 36
7.3 Time-Sharing . . . 36
7.4 Il ruolo dello scheduler . . . 37
7.5 Numero di processi in esecuzione . . . 37
7.6 Processi in attesa quando i core sono occupati . . . 38
7.7 Grado di multiprogrammazione . . . 38
7.8 Processi I/O-bound . . . 38
7.9 Processi CPU-bound . . . 39
7.10 Osservazione conclusiva . . . 39

8 Code dei Processi e Transizioni **40**
8.1 Introduzione 40
8.2 Ready Queue 40
8.3 Wait Queues 40
8.4 Ciclo di vita dei processi e transizioni tra code 41
8.5 Osservazioni sulle code dei processi 41
8.6 Conclusione 42

9 Scheduling della CPU e Swapping **42**
9.1 Introduzione 42
9.2 Selezione dei processi da parte dello scheduler 42
9.3 Comportamento dei processi I/O-bound e CPU-bound 43
9.4 Preemption e condivisione equa della CPU 43
9.5 Swapping dei processi 43
9.6 Osservazioni conclusive 44

10 Context Switching **44**
10.1 Introduzione 44
10.2 Ruolo delle interruzioni 44
10.3 Salvataggio del contesto del processo corrente 45
10.4 Caricamento del contesto del nuovo processo 45
10.5 Definizione di context switch 45
10.6 Overhead del context switching 46
10.7 Osservazioni conclusive 46

11 Creazione dei Processi e Struttura ad Albero **47**
11.1 Introduzione 47
11.2 Creazione di processi figli 47
11.3 Struttura ad albero dei processi 48
11.4 Identificatori dei processi 48
11.5 Il processo *init* in Linux 48
11.6 Osservazione conclusiva 49

12 Allocazione delle Risorse ai Processi Figli **49**
12.1 Introduzione 49
12.2 Origine delle risorse 49
12.3 Condivisione e divisione delle risorse 50
12.4 Limitazione delle risorse del figlio 50
12.5 Osservazioni conclusive 50

13 Comportamento dei Processi Padre e Figlio **51**
13.1 Introduzione 51
13.2 Opzioni di esecuzione 51
13.3 Opzioni di spazio di indirizzamento 51
13.4 Osservazioni conclusive 52

14 Creazione dei Processi in UNIX **52**
14.1 Introduzione 52
14.2 La chiamata `fork()` 52
14.2.1 Valore di ritorno di `fork()` 52
14.3 La chiamata `exec()` 53
14.4 Opzioni del processo padre 53
14.5 Osservazioni conclusive 53

15 Terminazione dei Processi **54**
15.1 Introduzione 54
15.2 Terminazione volontaria del processo 54
15.3 Recupero dello stato del processo figlio 54
15.4 Terminazione forzata da parte del processo padre 54
15.5 Terminazione a cascata 55
15.6 Osservazioni conclusive 55

16 Terminazione dei Processi e Processi Zombie **56**
16.1 Terminazione dei processi figli 56
16.2 Processo zombie 56
16.3 Rimozione del processo zombie 56
16.4 Osservazioni conclusive 57

17 Processi Indipendenti vs Processi Cooperanti **57**
17.1 Introduzione 57
17.2 Processi indipendenti 57
17.3 Processi cooperanti 57
17.4 Perché la cooperazione è importante 58
17.5 Osservazioni conclusive 58

18 Comunicazione tra Processi (IPC) **59**
18.1 Introduzione 59
18.2 Modello a Memoria Condivisa 59
18.3 Modello a Scambio di Messaggi 59
18.4 Confronto tra i due modelli 60
18.5 Osservazioni conclusive 60

19 Sistemi a Memoria Condivisa **61**
19.1 Introduzione 61
19.2 Attacco della regione condivisa 61
19.3 Gestione della memoria condivisa 61
19.4 Controllo della concorrenza 61
19.5 Osservazioni conclusive 61

20 Problema del Produttore-Consumatore **62**
20.1 Introduzione 62
20.2 Descrizione del problema 62
20.3 Approcci principali 62
20.3.1 Buffer illimitato (Unbounded Buffer) 62
20.3.2 Buffer limitato (Bounded Buffer) 62

20.4 Osservazioni conclusive 63

21 Scambio di Messaggi nella Comunicazione tra Processi (IPC) 63
21.1 Introduzione 63
21.2 Operazioni di base 63
21.3 Tipi di messaggi 63
21.4 Metodi di comunicazione 64
21.5 Osservazioni conclusive 64

22 Naming e Comunicazione Diretta/Indiretta 65
22.1 Introduzione 65
22.2 Comunicazione Diretta 65
22.2.1 Primitive 65
22.2.2 Caratteristiche 65
22.2.3 Varianti 65
22.2.4 Limitazioni 65
22.3 Comunicazione Indiretta (Mailbox) 66
22.3.1 Primitive 66
22.3.2 Caratteristiche 66
22.3.3 Problemi e soluzioni 66
22.4 Osservazioni conclusive 66

23 Sincronizzazione nella Comunicazione tra Processi 67
23.1 Introduzione 67
23.2 Tipi di invio (Send) 67
23.3 Tipi di ricezione (Receive) 67
23.4 Comportamento combinato 67
23.5 Esempio: Problema del Produttore-Consumatore 67
23.6 Osservazioni conclusive 68

24 Buffering nella Comunicazione tra Processi 68
24.1 Introduzione 68
24.2 Tipi di buffer 68
24.2.1 Capacità zero (Zero Capacity) 68
24.2.2 Capacità limitata (Bounded Capacity) 68
24.2.3 Capacità illimitata (Unbounded Capacity) 68
24.3 Idea chiave 69
24.4 Osservazioni conclusive 69

25 Socket e Comunicazione di Rete 70
25.1 Introduzione 70
25.2 Definizione di Socket 70
25.3 Porte di comunicazione 70
25.4 Esempio di comunicazione client–server 70
25.5 Idea chiave 70
25.6 Osservazioni conclusive 71

26 Remote Procedure Call (RPC) **71**
26.1 Introduzione 71
26.2 Meccanismo di funzionamento 71
26.3 Porte di servizio 71
26.4 Idea chiave 72
26.5 Osservazioni conclusive 72

27 Thread **73**
27.1 Introduzione 73
27.2 Struttura di un thread 73
27.3 Condivisione tra thread dello stesso processo 73
27.4 Tipi di processi in base ai thread 73
27.5 Idea chiave 73
27.6 Motivazioni per il multithreading 74
27.7 Esempi di applicazioni multithreaded 74
27.8 Vantaggi nei sistemi multicore 74
27.9 Osservazioni conclusive 74

28 Processi vs Thread in un Server Web **75**
28.1 Introduzione 75
28.2 Server a singolo processo 75
28.3 Server multithreaded 75
28.4 Idea chiave 75
28.5 Osservazioni conclusive 75

29 Vantaggi del Multithreading **76**
29.1 Introduzione 76
29.2 Principali vantaggi 76
29.3 Osservazioni conclusive 76

30 Sistemi Multicore e Multithreading **77**
30.1 Introduzione 77
30.2 Caratteristiche dei sistemi multicore 77
30.3 Multithreading nei sistemi multicore 77
30.4 Idea chiave 77
30.5 Osservazioni conclusive 77

31 Concorrenza vs. Parallelismo **78**
31.1 Introduzione 78
31.2 Possibili scenari 78
31.3 Tipi di parallelismo 78
31.3.1 Parallelismo sui dati (Data Parallelism) 78
31.3.2 Parallelismo sui task (Task Parallelism) 78
31.4 Idea chiave 78
31.5 Osservazioni conclusive 79

32 Thread a Livello Utente vs Thread a Livello Kernel **79**
32.1 Introduzione . . . 79
32.2 Modelli comuni di mapping . . . 79
32.3 Modello Many-to-One . . . 79
32.3.1 Limitazioni . . . 79
32.3.2 Risultato . . . 80
32.4 Modello One-to-One . . . 80
32.4.1 Svantaggi . . . 80
32.4.2 Idea chiave . . . 80
32.5 Modello Many-to-Many . . . 80
32.5.1 Vantaggi . . . 80
32.5.2 Svantaggi . . . 80
32.6 Idea chiave . . . 81
32.7 Osservazioni conclusive . . . 81

33 Modello a Due Livelli (Two-Level Model) **81**
33.1 Introduzione . . . 81
33.2 Funzionamento del modello . . . 81
33.3 Vantaggi principali . . . 81
33.4 Idea chiave . . . 82
33.5 Osservazioni conclusive . . . 82

34 Librerie per Thread **82**
34.1 Introduzione . . . 82
34.2 Implementazione delle librerie . . . 82
34.3 Principali librerie per thread . . . 83
34.4 Esempio di utilizzo . . . 83
34.5 Osservazioni conclusive . . . 83

35 Pthreads **84**
35.1 Introduzione . . . 84
35.2 Concetti chiave . . . 84
35.3 Esempio pratico . . . 84
35.3.1 Strategia Fork-Join . . . 84
35.4 Note sull'utilizzo multicore . . . 84
35.5 Osservazioni conclusive . . . 85

36 Thread su Windows **85**
36.1 Introduzione . . . 85
36.2 Creazione dei thread . . . 85
36.3 Sincronizzazione dei thread . . . 85
36.4 Idea chiave . . . 86
36.5 Osservazioni conclusive . . . 86

37 Thread in Java **86**
37.1 Introduzione . . . 86
37.2 Modalità di creazione dei thread . . . 86
37.3 Passaggi per la creazione di un thread . . . 86
37.4 Nota importante . . . 87

37.5 Osservazioni conclusive . . . 87

38 Modello Fork-Join **88**
38.1 Introduzione . . . 88
38.2 Funzionamento del Fork-Join . . . 88
38.3 Idea chiave . . . 88
38.4 Uso di `fork()` e `exec()` in programmi multithreaded . . . 88
38.5 Idea chiave aggiuntiva . . . 88
38.6 Osservazioni conclusive . . . 89

39 Gestione dei Segnali **89**
39.1 Introduzione . . . 89
39.2 Tipologie di segnali . . . 89
39.3 Processo di gestione dei segnali . . . 89
39.4 Tipi di gestori (handlers) . . . 89
39.5 Opzioni di gestione . . . 89
39.6 Idea chiave . . . 90
39.7 Osservazioni conclusive . . . 90

40 Gestione dei Segnali in Sistemi Multithreaded **90**
40.1 Introduzione . . . 90
40.2 Comportamento dei segnali . . . 90
40.3 Opzioni per l'invio dei segnali . . . 90
40.4 Gestione in UNIX . . . 90
40.5 Gestione in Windows . . . 91
40.6 Idea chiave . . . 91
40.7 Osservazioni conclusive . . . 91

41 Cancellazione dei Thread **92**
41.1 Introduzione . . . 92
41.2 Tipi di cancellazione . . . 92
41.3 Problematiche principali . . . 92
41.4 Cancellazione differita . . . 92
41.5 Idea chiave . . . 92
41.6 Osservazioni conclusive . . . 92

42 Dati Specifici del Thread (Thread-Specific Data, TLS) **93**
42.1 Introduzione . . . 93
42.2 Differenza rispetto alle variabili locali . . . 93
42.3 Supporto dei TLS . . . 93
42.4 Idea chiave . . . 93
42.5 Osservazioni conclusive . . . 93

43 Comunicazione tra Thread a Livello Kernel e Utente **94**
43.1 Introduzione . . . 94
43.2 Lightweight Process (LWP) . . . 94
43.3 Scheduler Activations . . . 94
43.4 Nota importante . . . 94
43.5 Idea chiave . . . 94

43.6 Osservazioni conclusive 95

1 Processi nei Sistemi Operativi Moderni

1.1 Introduzione generale

Nei sistemi operativi moderni, il concetto di **processo** rappresenta uno degli elementi fondamentali per comprendere come un calcolatore esegue programmi e gestisce le risorse del sistema. Un sistema operativo non si limita infatti a caricare ed eseguire un programma, ma deve coordinare l'esecuzione simultanea di molteplici attività, garantendo efficienza, sicurezza e isolamento tra le diverse componenti del sistema.

Per comprendere la necessità del concetto di processo, è utile considerare brevemente l'evoluzione storica dei sistemi di calcolo.

1.2 Sistemi iniziali: esecuzione di un solo programma alla volta

I primi sistemi informatici erano progettati per eseguire **un solo programma alla volta**. In questo modello di esecuzione:

- un programma veniva caricato in memoria
- il processore eseguiva esclusivamente le istruzioni di quel programma
- il sistema rimaneva dedicato a quel singolo compito fino al completamento dell'esecuzione

Questo modello, sebbene semplice, presentava numerosi limiti:

- il processore rimaneva spesso inutilizzato durante operazioni di input/output
- non era possibile eseguire più attività contemporaneamente
- le risorse hardware venivano sfruttate in modo inefficiente

Con il progresso tecnologico e con la crescente richiesta di sistemi più efficienti e reattivi, si è reso necessario sviluppare meccanismi che permettessero l'esecuzione contemporanea di più programmi.

1.3 Sistemi moderni: esecuzione concorrente di più programmi

I sistemi operativi moderni sono progettati per eseguire **più programmi contemporaneamente**. Questo paradigma è noto come **esecuzione concorrente**.

L'esecuzione concorrente non implica necessariamente che più programmi vengano eseguiti nello stesso identico istante (anche se ciò può accadere nei sistemi multicore), ma piuttosto che il sistema operativo alterni rapidamente l'esecuzione tra diversi programmi.

Questo meccanismo è alla base dei sistemi:

- **multitasking**
- **time-sharing**
- **multiprocessing**

Grazie a questi meccanismi, il sistema operativo può:

- mantenere il processore costantemente occupato
- gestire contemporaneamente molte applicazioni
- fornire all'utente l'impressione che più programmi siano in esecuzione nello stesso momento

Per rendere possibile questa gestione complessa delle attività, i sistemi operativi introducono un concetto astratto fondamentale: il **processo**.

1.4 Il concetto di processo

Un **processo** può essere definito come:

> un programma in esecuzione.

Questa definizione, sebbene sintetica, racchiude un significato molto profondo. Un programma, infatti, è semplicemente una sequenza statica di istruzioni memorizzata su un supporto di memorizzazione (ad esempio un disco). Quando il programma viene caricato in memoria ed eseguito dal processore, esso diventa un **processo**.

Un processo quindi non è solamente il codice del programma, ma include anche:

- lo stato corrente dell'esecuzione
- i dati associati al programma
- le risorse assegnate dal sistema operativo
- il contesto del processore

Tra le informazioni che caratterizzano un processo troviamo ad esempio:

- il **program counter**, che indica la prossima istruzione da eseguire
- i **registri della CPU**
- lo **stack** del processo
- la **memoria allocata**
- le **risorse di input/output**

Il sistema operativo mantiene tutte queste informazioni all'interno di strutture dati specifiche che consentono di gestire l'esecuzione dei processi.

1.5 Unità fondamentale di lavoro nei sistemi time-sharing

Nei sistemi di **time-sharing**, il processo rappresenta la **unità fondamentale di lavoro**. Questo significa che tutte le attività eseguite dal sistema operativo vengono organizzate e gestite sotto forma di processi.

Il time-sharing è una tecnica che permette a più utenti o programmi di condividere la CPU. Il sistema operativo assegna a ciascun processo un breve intervallo di tempo di esecuzione, chiamato generalmente:

time slice oppure quantum

Durante questo intervallo, il processo può utilizzare il processore. Al termine del tempo assegnato, il sistema operativo può sospendere il processo e assegnare la CPU ad un altro processo.

Questo meccanismo permette:

- una distribuzione equa delle risorse
- una migliore reattività del sistema
- la gestione simultanea di numerosi programmi

In questo contesto, il sistema operativo deve mantenere costantemente sotto controllo:

- lo stato di ogni processo
- le risorse utilizzate
- la pianificazione dell'esecuzione

1.6 Composizione di un sistema: processi del sistema operativo e processi utente

Un sistema operativo moderno è composto da due grandi categorie di processi.

1.6.1 Processi del sistema operativo

I **processi del sistema operativo** sono responsabili dell'esecuzione del codice interno del sistema operativo stesso.

Questi processi gestiscono funzioni fondamentali come:

- gestione della memoria
- gestione dei dispositivi di input/output
- pianificazione dei processi
- gestione del file system

Essi operano generalmente con privilegi elevati, poiché devono avere accesso diretto alle risorse hardware del sistema.

1.6.2 Processi utente

I **processi utente** eseguono invece il codice delle applicazioni utilizzate dagli utenti, come ad esempio:

- editor di testo
- browser web
- compilatori
- applicazioni scientifiche

Questi processi operano normalmente in una modalità meno privilegiata, chiamata spesso **user mode**, per evitare che eventuali errori o comportamenti malevoli possano compromettere l'intero sistema.

1.7 Idea fondamentale: concorrenza, controllo e isolamento

L'esecuzione concorrente di più processi introduce numerose complessità. Se più programmi vengono eseguiti contemporaneamente, è necessario garantire che:

- i processi non interferiscano tra loro
- le risorse vengano utilizzate correttamente
- i dati di un processo non vengano corrotti da un altro

Per questo motivo, due principi fondamentali guidano la progettazione dei sistemi operativi moderni:

1. **Controllo rigoroso dell'esecuzione dei processi**

   Il sistema operativo deve supervisionare costantemente l'esecuzione dei processi, decidendo quando un processo può utilizzare la CPU, quando deve essere sospeso e quali risorse può utilizzare.

2. **Isolamento dei processi**

   Ogni processo deve essere isolato dagli altri. Questo significa che:

   - un processo non può accedere direttamente alla memoria di un altro processo
   - eventuali errori in un processo non devono compromettere l'intero sistema
   - le risorse devono essere protette da accessi non autorizzati

L'insieme di questi meccanismi permette ai sistemi operativi moderni di garantire:

- stabilità
- sicurezza
- efficienza nell'utilizzo delle risorse

Il concetto di processo costituisce quindi la base su cui vengono costruiti molti altri meccanismi fondamentali dei sistemi operativi, tra cui la **pianificazione dei processi**, la **gestione della memoria**, la **sincronizzazione** e la **comunicazione tra processi**.

2 Struttura della Memoria di un Processo

2.1 Introduzione alla memoria di un processo

Quando un programma viene eseguito, il sistema operativo crea un **processo**. Oltre al codice eseguibile, il processo necessita di uno spazio di memoria organizzato in modo strutturato per poter gestire:

- il codice del programma
- le variabili globali
- le variabili locali
- le chiamate di funzione
- l'allocazione dinamica della memoria

Per questo motivo, lo spazio di memoria associato a un processo viene suddiviso in diverse **sezioni logiche**, ognuna con un ruolo specifico durante l'esecuzione del programma.

Questa organizzazione consente al sistema operativo e al compilatore di gestire in modo efficiente:

- l'esecuzione delle istruzioni
- l'accesso ai dati
- la gestione delle chiamate di funzione
- l'allocazione dinamica della memoria

Le principali sezioni della memoria di un processo sono:

- sezione **Text**
- sezione **Data**
- **Heap**
- **Stack**

Ognuna di queste aree ha proprietà diverse in termini di contenuto, dimensione e comportamento durante l'esecuzione del programma.

2.2 Sezioni della memoria

2.2.1 Sezione Text

La sezione **Text** contiene il **codice eseguibile** del programma.

Più precisamente, questa sezione include:

- le istruzioni macchina generate dal compilatore
- le istruzioni che la CPU deve eseguire

Questa sezione ha generalmente le seguenti caratteristiche:

- dimensione **fissa**
- contenuto **immutabile durante l'esecuzione**
- spesso marcata come **read-only** dal sistema operativo

La dimensione è considerata fissa perché il codice di un programma non cambia durante l'esecuzione. Una volta caricato in memoria dal sistema operativo, il codice rimane invariato fino alla terminazione del processo.

Proteggere questa sezione come memoria di sola lettura è importante per motivi di sicurezza e stabilità: impedisce infatti che il programma modifichi accidentalmente o intenzionalmente le proprie istruzioni.

2.2.2 Sezione Data

La sezione **Data** contiene le **variabili globali** e le **variabili statiche** del programma.

In particolare, questa sezione include:

- variabili globali dichiarate nel programma
- variabili statiche definite all'interno di funzioni
- eventuali costanti globali

Anche questa sezione ha una dimensione generalmente **fissa**, determinata al momento della compilazione e del caricamento del programma in memoria.

Questo accade perché il numero e la dimensione delle variabili globali sono noti già in fase di compilazione.

Durante l'esecuzione del programma:

- il contenuto delle variabili può cambiare
- ma la dimensione totale della sezione rimane invariata

Questa caratteristica distingue la sezione Data dalle aree di memoria dinamica come heap e stack.

2.2.3 Heap

L'**Heap** è la regione di memoria dedicata alla **allocazione dinamica della memoria**.

La memoria dinamica viene utilizzata quando il programma ha bisogno di allocare memoria la cui dimensione non è nota a priori al momento della compilazione.

Ad esempio, in molti linguaggi di programmazione si utilizzano funzioni come:

`malloc(), calloc(), new`

per richiedere memoria durante l'esecuzione del programma.

Le caratteristiche principali dell'heap sono:

- dimensione **variabile**
- crescita e riduzione durante l'esecuzione
- gestione esplicita o automatica della memoria

Quando il programma richiede nuova memoria, il sistema di gestione dell'heap **espande** questa regione. Quando la memoria non è più necessaria e viene liberata (ad esempio tramite `free()` o meccanismi di garbage collection), l'heap può **contrarsi**.

Formalmente possiamo descrivere questo comportamento come:

Heap può crescere oppure ridursi durante l'esecuzione del processo

L'uso corretto dell'heap è fondamentale per evitare problemi come:

- **memory leaks**
- **dangling pointers**
- frammentazione della memoria

2.2.4 Stack

Lo **Stack** è la struttura di memoria utilizzata per gestire:

- le chiamate di funzione
- le variabili locali
- gli indirizzi di ritorno delle funzioni

Lo stack è organizzato secondo il principio **LIFO** (Last In, First Out), cioè l'ultimo elemento inserito è il primo a essere rimosso.

Ogni volta che una funzione viene chiamata, il sistema crea una struttura chiamata:

record di attivazione (activation record)

Questo record contiene tutte le informazioni necessarie per l'esecuzione della funzione, tra cui:

- parametri della funzione
- variabili locali
- indirizzo di ritorno
- eventuali registri salvati

Anche lo stack ha una dimensione **variabile**, poiché il numero di chiamate di funzione attive può cambiare durante l'esecuzione del programma.

Pertanto lo stack può:

- crescere quando vengono chiamate nuove funzioni
- ridursi quando le funzioni terminano

2.3 Comportamento durante l'esecuzione

Per comprendere meglio il funzionamento di stack e heap è necessario analizzare cosa accade durante l'esecuzione di un programma.

2.3.1 Chiamata di funzione

Quando il programma effettua una **chiamata di funzione**, il sistema deve memorizzare tutte le informazioni necessarie per poter tornare correttamente al punto di esecuzione precedente.

Per questo motivo viene creato un **activation record** (record di attivazione), che viene inserito nello stack.

Formalmente possiamo descrivere questo processo come:

$$\text{Chiamata di funzione} \quad \Rightarrow \quad \text{inserimento di un activation record nello stack}$$

Questo record contiene:

- i parametri della funzione
- lo spazio per le variabili locali
- l'indirizzo di ritorno
- informazioni sul contesto di esecuzione

L'inserimento del record nello stack viene spesso chiamato:

$$\text{push dello stack}$$

2.3.2 Ritorno da una funzione

Quando la funzione termina la propria esecuzione, il sistema deve:

- rimuovere il record di attivazione
- ripristinare il contesto precedente
- continuare l'esecuzione dal punto corretto del programma

Questo processo può essere descritto come:

$$\text{Return} \quad \Rightarrow \quad \text{rimozione del record di attivazione dallo stack}$$

L'operazione di rimozione è detta:

$$\text{pop dello stack}$$

Grazie a questo meccanismo, lo stack cresce e si riduce dinamicamente durante l'esecuzione del programma.

2.3.3 Espansione e contrazione dell'heap

L'heap, a differenza dello stack, è utilizzato per memorizzare dati che devono persistere oltre la durata di una singola chiamata di funzione.

Quando il programma richiede nuova memoria dinamica, il gestore dell'heap esegue un'operazione di **allocazione**.

Questo comporta:

$$\text{Allocazione dinamica} \quad \Rightarrow \quad \text{espansione dell'heap}$$

Quando la memoria non è più necessaria e viene liberata tramite una funzione di deallocazione, si verifica invece:

$$\text{Deallocazione} \quad \Rightarrow \quad \text{contrazione dell'heap}$$

Questo comportamento dinamico consente ai programmi di adattare l'utilizzo della memoria alle proprie esigenze durante l'esecuzione.

2.4 Osservazione finale

La suddivisione della memoria di un processo nelle sezioni **text**, **data**, **heap** e **stack** costituisce uno dei principi fondamentali dell'architettura dei sistemi operativi moderni.

Questa organizzazione permette di:

- separare codice e dati
- gestire in modo efficiente la memoria
- supportare l'esecuzione di programmi complessi
- garantire maggiore sicurezza e stabilità del sistema

Comprendere questa struttura è essenziale per analizzare il comportamento dei programmi, il funzionamento delle chiamate di funzione e la gestione dinamica della memoria nei sistemi operativi.

3 Programma e Processo

3.1 Introduzione

Per comprendere in modo corretto il funzionamento dei sistemi operativi moderni è fondamentale distinguere chiaramente tra i concetti di **programma** e **processo**. Sebbene nel linguaggio quotidiano questi due termini vengano spesso utilizzati come sinonimi, dal punto di vista dei sistemi operativi essi rappresentano due entità concettualmente molto diverse.

La distinzione tra programma e processo è cruciale perché permette di comprendere:

- come i programmi vengono eseguiti dal sistema operativo
- come il sistema gestisce più esecuzioni simultanee dello stesso programma
- come vengono allocate e isolate le risorse di sistema

In questa sezione analizzeremo nel dettaglio il significato di questi due concetti e la relazione che li collega.

3.2 Il programma

Un **programma** può essere definito come una **entità passiva**.

Più precisamente, un programma è un **file contenente una sequenza di istruzioni** che descrivono un certo comportamento computazionale. Questo file è memorizzato su un dispositivo di memoria permanente, come ad esempio:

- un disco rigido
- un'unità SSD
- un supporto di archiviazione di rete

Il programma contiene tipicamente:

- il codice eseguibile
- eventuali dati statici
- informazioni necessarie al sistema per l'esecuzione

Dal punto di vista concettuale, il programma è considerato **passivo** perché:

- non è in esecuzione
- non utilizza risorse della CPU
- non occupa spazio nella memoria principale del sistema

In altre parole, il programma rappresenta semplicemente una **descrizione statica** di un insieme di istruzioni che potrebbero essere eseguite, ma che non sono ancora state attivate.

3.3 Il processo

Un **processo**, al contrario, è una **entità attiva**.

Un processo rappresenta infatti un **programma in esecuzione**. Quando il sistema operativo avvia l'esecuzione di un programma, crea una nuova entità dinamica che include:

- il codice del programma
- lo stato corrente dell'esecuzione
- le risorse assegnate dal sistema operativo
- lo spazio di memoria del processo

Pertanto, un processo non è semplicemente il codice del programma, ma una struttura molto più complessa che comprende tutte le informazioni necessarie per gestire l'esecuzione del programma.

Tra queste informazioni troviamo, ad esempio:

- il **program counter**, che indica la prossima istruzione da eseguire
- i **registri della CPU**
- la memoria assegnata al processo
- lo stato del processo (in esecuzione, in attesa, sospeso, ecc.)

Il processo è quindi una entità **dinamica**, poiché il suo stato evolve continuamente durante l'esecuzione.

3.4 Quando un programma diventa un processo

Una domanda fondamentale è la seguente:

Quando un programma diventa un processo?

La trasformazione di un programma in processo avviene nel momento in cui il sistema operativo **carica l'eseguibile in memoria** e avvia la sua esecuzione.

Questo processo può essere descritto attraverso i seguenti passaggi:

1. L'utente o il sistema richiede l'esecuzione di un programma.
2. Il sistema operativo individua il file eseguibile sul dispositivo di memoria.
3. Il sistema operativo carica il codice del programma nella memoria principale.
4. Viene creata una struttura dati interna che rappresenta il processo.
5. Il sistema operativo inizializza lo stato del processo e prepara l'esecuzione.

Formalmente possiamo esprimere questo concetto nel modo seguente:

$$\text{Programma su disco} \longrightarrow \text{Caricamento in memoria} \longrightarrow \text{Processo}$$

Il momento esatto in cui l'eseguibile viene caricato in memoria segna quindi la transizione da entità passiva (programma) a entità attiva (processo).

3.5 Più processi derivati dallo stesso programma

Un aspetto molto importante dei sistemi operativi moderni è la possibilità di creare **più processi a partire dallo stesso programma**.

Questo significa che lo stesso file eseguibile può essere avviato più volte contemporaneamente, generando diversi processi indipendenti tra loro.

Un esempio molto comune è rappresentato dai **browser web**. Un utente può aprire più finestre o più istanze dello stesso browser, e ognuna di esse verrà gestita dal sistema operativo come un processo distinto.

3.5.1 Condivisione della sezione di testo

Quando più processi derivano dallo stesso programma, essi possono condividere alcune parti della memoria.

In particolare, la **sezione text**, che contiene il codice eseguibile, può essere condivisa tra i processi.

Questo è possibile perché:

- il codice del programma è immutabile
- nessun processo modifica le istruzioni
- più processi possono eseguire le stesse istruzioni senza interferire tra loro

La condivisione della sezione text consente di ridurre il consumo di memoria, poiché il codice del programma non deve essere duplicato per ogni processo.

3.5.2 Separazione delle altre sezioni di memoria

Sebbene il codice possa essere condiviso, ogni processo deve avere le proprie strutture dati indipendenti.

Pertanto, ciascun processo possiede una copia separata delle seguenti sezioni di memoria:

- sezione **data**
- **heap**
- **stack**

Questa separazione è necessaria perché ogni processo deve poter:

- modificare le proprie variabili
- eseguire le proprie chiamate di funzione
- allocare memoria dinamica

Se queste aree fossero condivise, i processi interferirebbero tra loro causando errori e comportamenti imprevedibili.

3.6 Esempio: più istanze di un browser

Consideriamo un esempio concreto per chiarire ulteriormente questo concetto.

Supponiamo che un utente apra tre finestre dello stesso browser web. Dal punto di vista del sistema operativo, questa situazione corrisponde alla creazione di tre processi distinti:

- Processo 1: prima istanza del browser
- Processo 2: seconda istanza del browser
- Processo 3: terza istanza del browser

In questa configurazione:

- il codice del browser può essere condiviso tra tutti i processi
- ogni processo mantiene i propri dati indipendenti
- ogni processo ha il proprio stack e il proprio heap

Questo garantisce che:

- un errore in una finestra non comprometta le altre
- i dati delle diverse sessioni rimangano separati
- il sistema utilizzi la memoria in modo efficiente

3.7 Osservazione conclusiva

La distinzione tra programma e processo è uno dei concetti fondamentali dei sistemi operativi. Possiamo riassumere questa differenza nel seguente modo:

Programma	**Processo**
Entità passiva	Entità attiva
File su disco	Programma in esecuzione
Contiene istruzioni	Contiene stato di esecuzione
Nessuna risorsa allocata	Risorse allocate dal sistema operativo

Comprendere questa distinzione è essenziale per lo studio di molti altri meccanismi dei sistemi operativi, tra cui la **gestione dei processi**, la **pianificazione della CPU**, la **sincronizzazione** e la **comunicazione tra processi**.

4 Stati di un Processo

4.1 Introduzione al concetto di stato di un processo

Durante la sua esecuzione, un processo non rimane sempre nella stessa condizione operativa. Al contrario, il suo comportamento evolve nel tempo in funzione delle operazioni che deve svolgere e delle decisioni prese dal sistema operativo.

Per descrivere questa evoluzione dinamica, i sistemi operativi introducono il concetto di **stato del processo**.

Lo **stato di un processo** rappresenta la condizione corrente del processo in un determinato istante della sua esecuzione. Tale stato indica cosa il processo sta facendo in quel momento oppure perché non sta utilizzando il processore.

Formalmente, possiamo considerare lo stato come una variabile logica associata a ogni processo che descrive la fase del suo ciclo di vita.

Il sistema operativo utilizza queste informazioni per:

- gestire la pianificazione della CPU
- decidere quale processo deve essere eseguito
- coordinare l'accesso alle risorse di sistema

In generale, un processo può trovarsi in uno dei seguenti stati fondamentali.

4.2 Nuovo (New)

Lo stato **Nuovo** rappresenta la fase iniziale del ciclo di vita di un processo.

Un processo si trova nello stato *New* quando il sistema operativo sta effettuando le operazioni necessarie per la sua creazione.

Queste operazioni includono, ad esempio:

- l'allocazione delle strutture dati interne del sistema operativo
- la creazione della struttura di controllo del processo
- l'allocazione iniziale della memoria
- l'inizializzazione delle informazioni di stato

In questa fase il processo non è ancora pronto per essere eseguito dal processore. Il sistema operativo deve infatti completare tutte le operazioni di preparazione prima di inserirlo tra i processi pronti all'esecuzione.

4.3 In esecuzione (Running)

Lo stato **Running** indica che il processo sta attualmente eseguendo le proprie istruzioni sulla CPU.

In questa condizione:

- il processore sta eseguendo istruzioni appartenenti al processo
- i registri della CPU contengono il contesto di esecuzione del processo
- il *program counter* indica la prossima istruzione del processo da eseguire

Questo stato rappresenta la fase in cui il processo utilizza attivamente la risorsa più critica del sistema: il **processore**.

È importante osservare che, in un sistema con una singola CPU, **solo un processo alla volta può trovarsi nello stato Running**. Il processore può infatti eseguire una sola sequenza di istruzioni per volta.

Nei sistemi multicore la situazione è leggermente diversa: possono essere in esecuzione contemporaneamente tanti processi quanti sono i core disponibili. Tuttavia, per ogni singolo core rimane valido il principio secondo cui può essere eseguito un solo processo alla volta.

4.4 In attesa (Waiting)

Lo stato **Waiting** (o **Blocked**) rappresenta la condizione in cui un processo non può continuare l'esecuzione perché deve attendere il verificarsi di un determinato evento.

Tra gli eventi più comuni che possono causare questa situazione troviamo:

- il completamento di un'operazione di input/output
- la ricezione di un segnale
- la disponibilità di una risorsa richiesta
- la conclusione di un'operazione asincrona

Ad esempio, se un processo richiede la lettura di dati da un disco, il sistema operativo avvia l'operazione di I/O e sospende temporaneamente il processo. Durante questo periodo il processo non può eseguire istruzioni e viene quindi posto nello stato di attesa.

Quando l'evento atteso si verifica, il sistema operativo può spostare il processo in uno stato che gli permetta nuovamente di competere per l'utilizzo della CPU.

4.5 Pronto (Ready)

Lo stato **Ready** indica che il processo è completamente preparato per l'esecuzione ma non sta ancora utilizzando il processore.

In altre parole:

- il processo possiede tutte le risorse necessarie per eseguire
- non è in attesa di alcun evento
- deve soltanto attendere che il sistema operativo gli assegni la CPU

I processi in stato Ready vengono generalmente inseriti in una struttura dati chiamata **ready queue**. Questa coda contiene tutti i processi pronti all'esecuzione.

Il **scheduler** del sistema operativo seleziona periodicamente uno dei processi presenti nella ready queue e gli assegna il processore.

Formalmente possiamo rappresentare questo passaggio come:

$$\text{Ready} \quad \longrightarrow \quad \text{Running}$$

quando il processo viene scelto dallo scheduler.

4.6 Terminato (Terminated)

Lo stato **Terminated** rappresenta la fase finale del ciclo di vita di un processo.

Un processo entra in questo stato quando la sua esecuzione è completamente terminata. Questo può avvenire per diversi motivi:

- il programma ha completato tutte le sue istruzioni
- il processo ha terminato con una chiamata esplicita di uscita
- il sistema operativo ha forzato la terminazione
- si è verificato un errore irreversibile

Quando un processo termina:

- il sistema operativo rilascia tutte le risorse assegnate
- la memoria occupata viene liberata
- le strutture di controllo del processo vengono rimosse

Dopo queste operazioni, il processo cessa definitivamente di esistere.

4.7 Osservazioni sui nomi degli stati

È importante sottolineare che i nomi degli stati possono variare tra diversi sistemi operativi.

Ad esempio:

- lo stato *Waiting* può essere chiamato *Blocked*
- alcune implementazioni distinguono più stati di attesa
- alcuni sistemi introducono stati intermedi aggiuntivi

Nonostante queste variazioni terminologiche, il modello concettuale di base rimane sostanzialmente lo stesso.

4.8 Numero di processi nei diversi stati

Nei sistemi operativi moderni possono esistere contemporaneamente molti processi in stati diversi.

In particolare:

- **solo un processo per CPU può trovarsi nello stato Running**
- **molti processi possono trovarsi nello stato Ready**
- **molti processi possono trovarsi nello stato Waiting**

Questo comportamento deriva dal fatto che la CPU è una risorsa limitata, mentre il sistema operativo può mantenere in memoria numerosi processi pronti o temporaneamente bloccati.

4.9 Osservazione conclusiva

Il modello degli stati dei processi costituisce uno strumento fondamentale per descrivere il ciclo di vita di un processo all'interno di un sistema operativo.

Attraverso questo modello, il sistema operativo può:

- monitorare l'evoluzione dei processi
- pianificare l'utilizzo della CPU
- gestire in modo efficiente le risorse del sistema

La comprensione degli stati dei processi è inoltre essenziale per lo studio di meccanismi più avanzati come la **pianificazione dei processi**, il **context switching** e la **gestione delle code di scheduling**.

5 Blocco di Controllo del Processo (Process Control Block)

5.1 Introduzione

Per poter gestire in modo efficace i processi in esecuzione, il sistema operativo deve mantenere una grande quantità di informazioni relative a ciascun processo. Queste informazioni sono necessarie per monitorare lo stato del processo, gestire le risorse assegnate e permettere al sistema di sospendere e riprendere l'esecuzione dei processi in qualsiasi momento.

A tal fine, i sistemi operativi utilizzano una struttura dati fondamentale chiamata **Process Control Block**, spesso abbreviata con l'acronimo **PCB**.

Il **Process Control Block** è una struttura dati mantenuta dal sistema operativo che contiene tutte le informazioni necessarie per descrivere e gestire un processo.

Ogni processo presente nel sistema è associato a un PCB distinto. Questo significa che il sistema operativo mantiene una collezione di PCB, uno per ciascun processo attivo.

Il PCB rappresenta quindi la principale struttura attraverso cui il sistema operativo tiene traccia dei processi e ne controlla l'esecuzione.

5.2 Informazioni contenute nel PCB

Il PCB contiene numerose informazioni riguardanti il processo. Queste informazioni possono essere raggruppate in diverse categorie principali.

5.2.1 Stato del processo

Il PCB memorizza lo **stato corrente del processo**. Come discusso nella sezione precedente, un processo può trovarsi in diversi stati, tra cui:

- *New*
- *Ready*
- *Running*
- *Waiting*
- *Terminated*

Questa informazione è fondamentale per il sistema operativo, poiché permette di determinare il comportamento del processo e le operazioni che devono essere eseguite su di esso.

Ad esempio, un processo nello stato *Ready* può essere selezionato dallo scheduler per essere eseguito, mentre un processo nello stato *Waiting* non può essere eseguito finché non si verifica l'evento atteso.

5.2.2 Program Counter

Il **Program Counter** (contatore di programma) è un registro speciale che indica l'indirizzo della prossima istruzione che il processo deve eseguire.

Nel PCB viene memorizzato il valore del program counter relativo al processo. Questa informazione è essenziale perché consente al sistema operativo di riprendere l'esecuzione del processo esattamente dal punto in cui era stata interrotta.

Quando il sistema operativo sospende temporaneamente un processo (ad esempio per assegnare la CPU a un altro processo), il valore corrente del program counter viene salvato nel PCB.

Successivamente, quando il processo verrà nuovamente eseguito, il valore del program counter verrà ripristinato, permettendo la ripresa corretta dell'esecuzione.

5.2.3 Registri della CPU

Oltre al program counter, il PCB deve memorizzare il contenuto di tutti i **registri della CPU** associati al processo.

I registri della CPU contengono informazioni temporanee utilizzate durante l'esecuzione delle istruzioni, come ad esempio:

- registri generali
- registri indice
- registri di stato
- registri di controllo

Quando il sistema operativo effettua un **context switch**, ovvero un cambio di processo in esecuzione, il contenuto dei registri della CPU viene salvato nel PCB del processo corrente.

Successivamente, quando il processo riprenderà l'esecuzione, i registri verranno ripristinati dal PCB.

Questo meccanismo garantisce che ogni processo possa continuare la propria esecuzione senza perdere informazioni sul proprio stato.

5.2.4 Informazioni di scheduling della CPU

Il PCB contiene anche informazioni necessarie per la **pianificazione della CPU** (CPU scheduling).

Tra queste informazioni troviamo, ad esempio:

- la **priorità del processo**
- i **puntatori alle code di scheduling**
- altri parametri utilizzati dagli algoritmi di scheduling

Il sistema operativo utilizza queste informazioni per decidere quale processo debba ricevere la CPU in un determinato momento.

Ad esempio, in sistemi basati su priorità, il valore della priorità memorizzato nel PCB può influenzare direttamente l'ordine con cui i processi vengono eseguiti.

5.2.5 Informazioni di gestione della memoria

Il PCB deve inoltre contenere informazioni relative alla **gestione della memoria** del processo.

Queste informazioni permettono al sistema operativo di determinare quali porzioni di memoria sono assegnate al processo e come devono essere interpretati gli indirizzi di memoria.

Tra le informazioni memorizzate troviamo:

- registri **base** e **limit**
- **tabelle delle pagine** (page tables)
- **tabelle dei segmenti** (segment tables)

Queste strutture sono fondamentali per implementare tecniche di gestione della memoria come:

- paginazione
- segmentazione
- memoria virtuale

Grazie a queste informazioni, il sistema operativo può tradurre gli indirizzi logici utilizzati dal processo negli indirizzi fisici della memoria.

5.2.6 Informazioni di contabilizzazione

Il PCB contiene anche informazioni utilizzate per la **contabilizzazione delle risorse** utilizzate dal processo.

Tra queste informazioni possono essere incluse:

- il tempo totale di CPU utilizzato dal processo
- eventuali limiti di utilizzo delle risorse
- identificatori del processo
- identificatori dell'utente proprietario del processo

Queste informazioni possono essere utilizzate per diversi scopi, tra cui:

- monitoraggio delle prestazioni del sistema
- controllo delle risorse utilizzate
- applicazione di politiche di sicurezza o di quota

5.2.7 Stato delle operazioni di Input/Output

Infine, il PCB contiene informazioni relative allo stato delle operazioni di **Input/Output** associate al processo.

Queste informazioni possono includere:

- dispositivi di I/O assegnati al processo
- elenco dei file aperti
- stato delle operazioni di I/O in corso

Questi dati permettono al sistema operativo di gestire correttamente le operazioni di input e output associate al processo.

Ad esempio, se un processo è in attesa del completamento di un'operazione di I/O, il sistema operativo può utilizzare le informazioni contenute nel PCB per determinare quando l'operazione è terminata e quando il processo può riprendere l'esecuzione.

5.3 Scopo del Process Control Block

Il PCB ha un ruolo fondamentale nel funzionamento del sistema operativo.

Il suo scopo principale è quello di **contenere tutte le informazioni necessarie per gestire e controllare un processo** durante l'intero ciclo di vita.

In particolare, il PCB permette al sistema operativo di:

- sospendere temporaneamente un processo
- salvare il suo stato di esecuzione
- riprendere successivamente l'esecuzione dal punto corretto

Questo meccanismo è alla base del **context switching**, ovvero la capacità del sistema operativo di alternare rapidamente l'esecuzione tra diversi processi.

Formalmente possiamo descrivere questo comportamento come:

$$\text{Salvataggio dello stato nel PCB} \quad \Rightarrow \quad \text{interruzione del processo}$$

$$\text{Ripristino dello stato dal PCB} \quad \Rightarrow \quad \text{ripresa dell'esecuzione}$$

Grazie al PCB, il sistema operativo può gestire simultaneamente numerosi processi, garantendo che ciascuno di essi mantenga il proprio stato e possa essere eseguito correttamente.

5.4 Osservazione conclusiva

Il **Process Control Block** rappresenta una delle strutture dati più importanti all'interno di un sistema operativo.

Senza il PCB sarebbe impossibile:

- gestire più processi contemporaneamente
- effettuare il context switching
- mantenere l'isolamento tra i processi

Il PCB costituisce quindi il meccanismo attraverso cui il sistema operativo mantiene il controllo completo sull'esecuzione dei processi e sulle risorse a essi associate.

6 Dai Processi ai Thread

6.1 Introduzione

Nel modello classico dei sistemi operativi, il concetto di **processo** rappresentava l'unità fondamentale di esecuzione. Ogni programma in esecuzione era associato a un singolo processo e, all'interno di tale processo, esisteva un'unica sequenza di istruzioni che veniva eseguita dalla CPU.

Questa sequenza di istruzioni è chiamata **flusso di esecuzione** o **execution path**. Nel modello tradizionale, quindi, esisteva una corrispondenza diretta tra processo e flusso di esecuzione.

Con l'aumentare della complessità delle applicazioni e con l'evoluzione dell'hardware, questo modello ha mostrato alcuni limiti. Per superare tali limitazioni, i sistemi operativi moderni hanno introdotto un concetto più flessibile: i **thread**.

I thread consentono di suddividere l'esecuzione di un processo in più flussi di esecuzione indipendenti ma cooperanti.

6.2 Modello tradizionale: un processo, un solo percorso di esecuzione

Nel modello tradizionale dei sistemi operativi si assumeva che:

$$\text{un processo} \quad = \quad \text{un unico percorso di esecuzione}$$

Questo significa che ogni processo possedeva un solo contatore di programma (*program counter*) e una sola sequenza di istruzioni eseguite in modo sequenziale.

In altre parole, l'intero comportamento del processo era determinato da un singolo flusso di controllo.

Questo modello può essere interpretato come:

$$\text{Processo} \rightarrow \text{Thread singolo}$$

dove il termine *thread* non veniva esplicitamente utilizzato, ma il concetto era implicitamente presente.

Sebbene questo approccio fosse semplice da implementare, esso presentava alcuni limiti significativi.

6.3 Limitazioni del modello tradizionale

Il principale limite del modello tradizionale è che esso **non consente il multitasking interno a un singolo processo**.

In altre parole, se un processo deve svolgere più attività contemporaneamente, nel modello tradizionale tali attività devono essere eseguite in modo strettamente sequenziale.

Questo può causare inefficienze significative. Consideriamo, ad esempio, un programma che deve:

- gestire l'interfaccia utente
- effettuare operazioni di rete
- eseguire calcoli complessi

Se esiste un solo flusso di esecuzione, queste attività devono essere eseguite una dopo l'altra. Se una di esse richiede molto tempo o rimane bloccata in attesa di un'operazione di input/output, l'intero processo rimane bloccato.

Questo comportamento può compromettere:

- la reattività delle applicazioni
- l'efficienza dell'utilizzo delle risorse
- l'esperienza dell'utente

Per affrontare questi problemi è stato introdotto il concetto di **thread multipli all'interno dello stesso processo**.

6.4 Supporto ai thread nei sistemi operativi moderni

I sistemi operativi moderni supportano la presenza di **più thread all'interno di uno stesso processo**.

Un **thread** può essere definito come una **unità di esecuzione leggera** appartenente a un processo.

Ogni thread possiede:

- il proprio program counter
- i propri registri della CPU
- il proprio stack

Tuttavia, tutti i thread appartenenti allo stesso processo condividono alcune risorse fondamentali, tra cui:

- lo spazio di indirizzamento del processo
- la sezione di codice
- i dati globali
- le risorse di input/output

Questa struttura consente ai thread di collaborare facilmente e di comunicare tra loro in modo efficiente.

Possiamo rappresentare concettualmente questa struttura come:

$$\text{Processo} \rightarrow \begin{cases} \text{Thread 1} \\ \text{Thread 2} \\ \text{Thread 3} \\ \vdots \end{cases}$$

6.5 Esecuzione simultanea di più attività

Grazie all'introduzione dei thread, un singolo processo può eseguire **più attività contemporaneamente**.

Ad esempio, un'applicazione può utilizzare thread distinti per:

- gestire l'interfaccia utente
- scaricare dati dalla rete
- elaborare informazioni in background

Questo approccio migliora significativamente la reattività delle applicazioni. Se un thread rimane bloccato in attesa di un'operazione di input/output, gli altri thread del processo possono continuare a eseguire le proprie attività.

In questo modo, il processo nel suo complesso rimane attivo e operativo.

6.6 Esecuzione parallela nei sistemi multicore

Nei sistemi dotati di **processori multicore**, l'utilizzo dei thread diventa ancora più potente.

In presenza di più core della CPU, il sistema operativo può assegnare thread diversi a core diversi, consentendo una vera e propria **esecuzione parallela**.

Formalmente possiamo rappresentare questa situazione come:

Core della CPU	Thread in esecuzione
Core 1	Thread A
Core 2	Thread B
Core 3	Thread C

Questo permette di sfruttare in modo più efficiente la potenza di calcolo disponibile.

Le applicazioni progettate per utilizzare più thread possono quindi ottenere significativi miglioramenti nelle prestazioni.

6.7 Estensione del PCB per supportare i thread

L'introduzione dei thread richiede alcune modifiche alle strutture dati utilizzate dal sistema operativo.

In particolare, il **Process Control Block (PCB)** deve essere esteso per includere informazioni relative ai thread appartenenti al processo.

Poiché ogni thread possiede un proprio stato di esecuzione, il sistema operativo deve mantenere informazioni specifiche per ciascun thread, come ad esempio:

- stato del thread
- program counter del thread
- registri della CPU associati al thread
- stack del thread

In molte implementazioni moderne, queste informazioni vengono memorizzate in una struttura separata chiamata **Thread Control Block (TCB)**, collegata al PCB del processo.

Possiamo rappresentare questa relazione nel modo seguente:

$$\text{PCB} \rightarrow \begin{cases} \text{TCB}_1 \\ \text{TCB}_2 \\ \text{TCB}_3 \end{cases}$$

dove ciascun TCB contiene le informazioni specifiche di un thread.

6.8 Osservazione conclusiva

Il passaggio dal modello basato esclusivamente sui processi al modello che include anche i thread rappresenta una delle evoluzioni più importanti nella progettazione dei sistemi operativi.

L'introduzione dei thread permette di:

- migliorare la reattività delle applicazioni
- sfruttare meglio le risorse hardware
- supportare il parallelismo nei sistemi multicore

Di conseguenza, nei sistemi operativi moderni i thread sono spesso considerati la **vera unità di esecuzione della CPU**, mentre il processo rappresenta principalmente il contenitore delle risorse condivise.

7 Multiprogrammazione e Time-Sharing

7.1 Introduzione

Uno degli obiettivi principali dei sistemi operativi moderni è utilizzare in modo efficiente le risorse hardware disponibili, in particolare la **CPU**. Nei primi sistemi informatici, il processore rimaneva spesso inattivo durante le operazioni di input/output, causando un notevole spreco di capacità computazionale.

Per migliorare l'efficienza del sistema sono stati introdotti due concetti fondamentali:

- **Multiprogrammazione**
- **Time-sharing**

Questi due meccanismi permettono al sistema operativo di gestire contemporaneamente più processi, aumentando l'utilizzo della CPU e migliorando l'esperienza degli utenti.

7.2 Multiprogrammazione

La **multiprogrammazione** è una tecnica progettata per **massimizzare l'utilizzo della CPU**.

L'idea di base è mantenere più processi in memoria contemporaneamente. Quando un processo non può continuare la propria esecuzione perché deve attendere il completamento di un'operazione di input/output, il sistema operativo assegna immediatamente la CPU a un altro processo.

Questo comportamento può essere rappresentato concettualmente nel seguente modo:

$$\text{Processo A (I/O)} \quad \Rightarrow \quad \text{CPU libera} \quad \Rightarrow \quad \text{Processo B eseguito}$$

In questo modo, il tempo in cui la CPU rimane inutilizzata viene drasticamente ridotto.

La multiprogrammazione consente quindi al sistema operativo di:

- mantenere la CPU occupata il più possibile
- migliorare l'efficienza complessiva del sistema
- gestire contemporaneamente più programmi

7.3 Time-Sharing

Il **time-sharing** è un'estensione della multiprogrammazione progettata per supportare sistemi interattivi con più utenti.

Nel time-sharing, il sistema operativo suddivide il tempo di utilizzo della CPU in piccoli intervalli chiamati:

$$\text{quantum di tempo} \quad \text{(time slice)}$$

Ogni processo riceve la CPU per un breve intervallo di tempo. Al termine di questo intervallo, il sistema operativo sospende temporaneamente il processo e assegna la CPU a un altro processo.

Questo meccanismo di alternanza avviene molto rapidamente, creando l'impressione che tutti i programmi siano in esecuzione simultaneamente.

Il time-sharing permette quindi di:

- supportare più utenti contemporaneamente
- mantenere il sistema reattivo
- garantire una distribuzione equa della CPU

7.4 Il ruolo dello scheduler

Per coordinare l'esecuzione dei processi, il sistema operativo utilizza un componente chiamato **scheduler**.

Lo scheduler ha il compito di selezionare quale processo deve essere eseguito dalla CPU in un determinato momento.

Il processo di selezione può essere rappresentato come:

$$\text{Ready Queue} \longrightarrow \text{Scheduler} \longrightarrow \text{Processo in esecuzione}$$

Lo scheduler utilizza diversi algoritmi per prendere questa decisione, considerando fattori come:

- priorità dei processi
- tempo di esecuzione
- comportamento di input/output

La scelta dell'algoritmo di scheduling influenza significativamente le prestazioni e la reattività del sistema.

7.5 Numero di processi in esecuzione

In un sistema con più processi attivi, non tutti i processi possono eseguire contemporaneamente.

Una regola fondamentale è la seguente:

> **Per ogni core della CPU può essere eseguito un solo processo alla volta.**

Questo significa che:

- in un sistema con una sola CPU può eseguire un solo processo
- in un sistema multicore possono eseguire simultaneamente più processi

Se il numero di processi pronti all'esecuzione supera il numero di core disponibili, alcuni processi devono attendere.

Questi processi vengono mantenuti nello stato **Ready** fino a quando uno dei core diventa disponibile.

7.6 Processi in attesa quando i core sono occupati

Quando tutti i core della CPU sono occupati, i processi aggiuntivi non possono essere eseguiti immediatamente.

In questo caso:

- i processi vengono inseriti nella **ready queue**
- attendono che uno dei processi in esecuzione rilasci la CPU

Questo meccanismo permette al sistema operativo di gestire un numero di processi molto maggiore rispetto al numero di core disponibili.

7.7 Grado di multiprogrammazione

Il **grado di multiprogrammazione** è definito come il numero di processi presenti contemporaneamente in memoria.

Formalmente possiamo esprimere questo concetto come:

$$\text{Grado di multiprogrammazione} = \text{numero di processi presenti in memoria}$$

Un grado di multiprogrammazione elevato significa che il sistema mantiene molti processi pronti o in attesa.

Questo può migliorare l'utilizzo della CPU, poiché aumenta la probabilità che esista sempre un processo pronto all'esecuzione.

Tuttavia, un numero troppo elevato di processi può anche causare problemi, come:

- maggiore overhead di gestione
- maggiore competizione per le risorse
- possibile degradazione delle prestazioni

Il sistema operativo deve quindi trovare un equilibrio ottimale.

7.8 Processi I/O-bound

I processi possono essere classificati in base al tipo di operazioni che eseguono più frequentemente.

Un **processo I/O-bound** è un processo che trascorre la maggior parte del proprio tempo eseguendo operazioni di input/output.

Questi processi:

- effettuano frequentemente richieste di I/O
- utilizzano la CPU solo per brevi periodi
- passano molto tempo nello stato di attesa

Esempi tipici di processi I/O-bound includono:

- applicazioni che leggono grandi quantità di dati da disco
- programmi che comunicano frequentemente tramite rete
- applicazioni interattive con l'utente

7.9 Processi CPU-bound

Un **processo CPU-bound** è invece un processo che utilizza intensivamente la CPU per eseguire calcoli.

Questi processi:

- eseguono lunghe sequenze di istruzioni
- effettuano poche operazioni di input/output
- mantengono la CPU occupata per periodi prolungati

Esempi di processi CPU-bound includono:

- simulazioni scientifiche
- elaborazione di immagini
- algoritmi di calcolo complesso

7.10 Osservazione conclusiva

La distinzione tra processi **I/O-bound** e **CPU-bound** è molto importante per la progettazione degli algoritmi di scheduling.

Un sistema efficiente dovrebbe essere in grado di bilanciare questi due tipi di processi in modo da:

- mantenere la CPU occupata
- ridurre i tempi di attesa
- migliorare la reattività del sistema

I concetti di multiprogrammazione e time-sharing rappresentano quindi la base del funzionamento dei sistemi operativi moderni e permettono la gestione efficiente di numerosi processi concorrenti.

8 Code dei Processi e Transizioni

8.1 Introduzione

Nei sistemi operativi, la gestione dei processi in memoria avviene tramite strutture chiamate **code dei processi** (*process queues*). Queste code permettono di organizzare i processi in base al loro stato e alle risorse di cui hanno bisogno, facilitando lo scheduling e la gestione delle transizioni tra stati.

L'utilizzo delle code è fondamentale per garantire:

- una corretta pianificazione dei processi
- l'efficiente utilizzo della CPU e delle risorse
- la gestione dei processi in attesa di eventi

8.2 Ready Queue

La **ready queue** è la struttura dati che contiene tutti i processi pronti all'esecuzione, cioè quei processi che:

- hanno tutte le risorse necessarie per l'esecuzione
- non sono in attesa di alcun evento
- devono attendere solo l'assegnazione della CPU

Dal punto di vista implementativo, la ready queue viene spesso realizzata come **lista collegata di PCB** (*linked list of Process Control Blocks*).

Ogni PCB memorizza tutte le informazioni necessarie per riprendere l'esecuzione del processo non appena lo scheduler gli assegna la CPU.

8.3 Wait Queues

Oltre alla ready queue, esistono le **wait queues**, cioè le code dei processi in attesa di eventi specifici.

I processi vengono inseriti in una wait queue quando:

- effettuano una richiesta di input/output
- devono attendere un segnale da un altro processo
- sono in attesa di una risorsa condivisa

Le wait queues consentono al sistema operativo di organizzare i processi in attesa in modo efficiente, facilitando la loro successiva ripresa.

8.4 Ciclo di vita dei processi e transizioni tra code

Il ciclo di vita di un processo può essere descritto come una sequenza di transizioni tra code e stati:

1. **Entrata nella ready queue:** quando un processo è creato o riprende l'esecuzione dopo un evento, viene inserito nella ready queue in attesa della CPU.
2. **Dispatching alla CPU:** lo scheduler seleziona un processo dalla ready queue e lo assegna al processore, passando lo stato del processo a *Running*.
3. **Blocco per operazioni di I/O:** se durante l'esecuzione il processo deve attendere il completamento di un evento (ad esempio un'operazione di input/output), viene spostato in una wait queue corrispondente all'evento.
4. **Interruzione:** se il tempo di esecuzione del processo termina o se un processo a priorità maggiore deve essere eseguito, il processo in esecuzione può essere interrotto e rimesso nella ready queue per attendere la CPU.
5. **Completamento:** quando il processo termina la sua esecuzione, viene rimosso dalla coda, e il PCB insieme alle risorse allocate viene liberato dal sistema operativo.

Queste transizioni possono essere rappresentate schematicamente come:

$$\text{Ready Queue} \xrightarrow{\text{dispatch}} \text{CPU (Running)} \xrightarrow{\text{I/O}} \text{Wait Queue} \xrightarrow{\text{evento completato}} \text{Ready Queue} \xrightarrow{\text{Completamento}} \text{term}$$

8.5 Osservazioni sulle code dei processi

Alcune considerazioni importanti riguardo all'utilizzo delle code:

- La gestione delle code è fondamentale per lo scheduling e per garantire che tutti i processi ricevano tempo di CPU in maniera equa.
- Le ready queue e le wait queue devono essere implementate in modo efficiente per minimizzare l'overhead del sistema operativo.
- Le transizioni tra code rappresentano le principali operazioni di gestione dei processi e sono strettamente legate allo stato dei processi.
- L'uso corretto delle code permette di mantenere alta l'efficienza della CPU e ridurre i tempi di inattività dei processi.

8.6 Conclusione

Le code dei processi e le transizioni tra di esse costituiscono la base del funzionamento dei sistemi operativi moderni.

Esse consentono di:

- organizzare i processi in base al loro stato
- gestire in modo efficiente l'allocazione della CPU
- coordinare l'esecuzione dei processi in attesa di eventi

Comprendere il funzionamento delle code dei processi è essenziale per affrontare lo studio di argomenti avanzati come:

- algoritmi di scheduling della CPU
- gestione delle risorse
- sincronizzazione e comunicazione tra processi

9 Scheduling della CPU e Swapping

9.1 Introduzione

Nei sistemi operativi moderni, i processi non rimangono statici nello stesso stato, ma **si spostano continuamente tra le code di ready e di attesa**. La gestione efficiente di questi spostamenti e l'assegnazione della CPU sono compiti fondamentali del sistema operativo, realizzati attraverso il **CPU scheduling**.

In questa sezione analizzeremo il ruolo dello scheduler, il comportamento dei processi in base al loro tipo, l'importanza della preemption e il concetto di **swapping** per la gestione della memoria.

9.2 Selezione dei processi da parte dello scheduler

Il **CPU scheduler** è il componente del sistema operativo responsabile di scegliere quale processo, tra quelli presenti nella *ready queue*, deve essere eseguito su un core della CPU.

Il funzionamento dello scheduler può essere descritto come segue:

1. Il sistema operativo mantiene una lista di processi pronti all'esecuzione.
2. Lo scheduler seleziona il processo più adatto secondo l'algoritmo di scheduling in uso (es. FIFO, Round-Robin, Priorità, ecc.).
3. Il processo selezionato viene assegnato a un core disponibile della CPU.
4. Il contesto del processo viene caricato dai PCB e il processo passa allo stato *Running*.

9.3 Comportamento dei processi I/O-bound e CPU-bound

I processi possono essere classificati in base al tipo di attività predominante:

- **Processi I/O-bound:** trascorrono la maggior parte del tempo in operazioni di input/output, con *brevi burst* di utilizzo della CPU. Ciò significa che richiedono spesso la CPU, ma per periodi molto brevi.
- **Processi CPU-bound:** effettuano lunghe elaborazioni computazionali con *lunghi burst* di utilizzo della CPU. Tendono quindi a occupare la CPU per periodi più estesi prima di richiedere operazioni di I/O.

La distinzione tra I/O-bound e CPU-bound è fondamentale per ottimizzare lo scheduling. Ad esempio, un algoritmo di scheduling può privilegiare processi I/O-bound per evitare che la CPU rimanga inattiva in attesa del completamento delle operazioni di I/O.

9.4 Preemption e condivisione equa della CPU

Nei sistemi con **preemption**, il sistema operativo può sospendere un processo in esecuzione prima che abbia terminato il suo burst di CPU, assegnando il processore a un altro processo.

La preemption è necessaria per:

- garantire una condivisione equa della CPU tra tutti i processi
- mantenere la reattività delle applicazioni interattive
- implementare algoritmi di scheduling basati su priorità o Round-Robin

Tipicamente, lo scheduler effettua queste decisioni **frequentemente**, ad esempio ogni 100 millisecondi o anche più rapidamente, a seconda del sistema e del carico di lavoro.

9.5 Swapping dei processi

Il **swapping** è una tecnica di gestione della memoria che consiste nel **rimuovere temporaneamente un processo dalla memoria principale** per liberare spazio e gestire meglio il carico del sistema.

Durante lo swapping:

- un processo viene trasferito su memoria secondaria (es. disco) se non può continuare l'esecuzione immediata.
- il sistema operativo può caricare un altro processo nella memoria principale, consentendone l'esecuzione.

Quando il processo swapato deve riprendere l'esecuzione, viene nuovamente caricato in memoria e reinserito nella ready queue.

Questa tecnica è utile quando:

- il numero di processi attivi supera la capacità della memoria fisica
- alcuni processi rimangono inattivi per periodi prolungati (es. in attesa di I/O)
- si vuole migliorare l'utilizzo della CPU senza aumentare la memoria fisica

9.6 Osservazioni conclusive

Lo scheduling della CPU e il swapping sono meccanismi strettamente interconnessi:

- Lo scheduler decide quale processo utilizzare la CPU, tenendo conto di priorità, stato e tipo di processo (I/O-bound o CPU-bound).
- La preemption garantisce che nessun processo monopolizzi la CPU e permette una distribuzione equa del tempo di elaborazione.
- Il swapping consente di adattare la memoria disponibile ai requisiti dei processi attivi, evitando blocchi e migliorando l'efficienza.

La combinazione di questi strumenti è essenziale per mantenere alte prestazioni, reattività e stabilità nei sistemi operativi moderni.

10 Context Switching

10.1 Introduzione

Il **context switching** è un meccanismo fondamentale dei sistemi operativi che consente di alternare l'esecuzione tra diversi processi sulla stessa CPU. Grazie al context switching, è possibile implementare la multiprogrammazione e il time-sharing, garantendo che più processi possano condividere la CPU in maniera efficiente e senza perdere il proprio stato di esecuzione.

Il context switching avviene quando il processore deve interrompere l'esecuzione di un processo corrente per eseguire un altro processo.

10.2 Ruolo delle interruzioni

Il context switch è generalmente innescato da un **interrupt**, ovvero un segnale che indica al sistema operativo che un evento richiede attenzione.

Gli eventi comuni che possono generare un interrupt includono:

- scadenza del quantum di tempo (nel time-sharing)
- completamento di un'operazione di I/O
- richieste di priorità da altri processi
- errori o eccezioni hardware/software

Quando si verifica un interrupt, il processore interrompe l'esecuzione del processo corrente e passa il controllo al sistema operativo, il quale si occupa di salvare lo stato del processo e selezionare il prossimo processo da eseguire.

10.3 Salvataggio del contesto del processo corrente

Durante il context switching, il sistema operativo deve salvare lo **stato del processo corrente** nel suo PCB. Questo stato comprende:

- i registri della CPU
- il program counter (indicante la prossima istruzione)
- le informazioni di stato del processo
- eventuali informazioni sulla gestione della memoria (base, limit, pagine, segmenti)

Il salvataggio accurato di queste informazioni è essenziale per garantire che il processo possa riprendere l'esecuzione esattamente dal punto in cui era stato interrotto, senza perdita di dati o corruzione dello stato.

10.4 Caricamento del contesto del nuovo processo

Dopo aver salvato il contesto del processo sospeso, la CPU carica il contesto del **prossimo processo da eseguire**, sempre prelevandolo dal relativo PCB.

Il caricamento del contesto comprende:

- ripristino dei registri della CPU
- impostazione del program counter
- ripristino dello stato di esecuzione

Una volta completato il caricamento, il nuovo processo entra nello stato *Running* e inizia a eseguire le proprie istruzioni sulla CPU.

10.5 Definizione di context switch

L'insieme di operazioni descritte — salvataggio del contesto di un processo, selezione del nuovo processo e caricamento del suo contesto — è noto come **context switch**.

Il context switch permette al sistema operativo di:

- sospendere temporaneamente un processo
- riprendere un altro processo
- mantenere l'illusione di esecuzione simultanea su un singolo core

10.6 Overhead del context switching

Il context switching introduce un **overhead**, cioè un tempo di elaborazione aggiuntivo non dedicato direttamente all'esecuzione dei processi utente.

Tipicamente, questo overhead è dell'ordine di pochi millisecondi. Tuttavia, la durata effettiva dipende da:

- complessità del sistema operativo
- quantità di registri e informazioni da salvare/caricare
- velocità dell'hardware

Sistemi operativi più complessi, con molte strutture dati e meccanismi di protezione, tendono ad avere tempi di switch maggiori rispetto a sistemi più semplici.

10.7 Osservazioni conclusive

Il context switching è un elemento centrale per:

- implementare la multiprogrammazione e il time-sharing
- consentire l'esecuzione concorrente di più processi
- garantire l'isolamento e la correttezza dello stato di ciascun processo

Comprendere il funzionamento del context switch è fondamentale per valutare:

- le prestazioni complessive del sistema
- l'efficienza della CPU
- l'impatto delle politiche di scheduling sui tempi di risposta

11 Creazione dei Processi e Struttura ad Albero

11.1 Introduzione

Nei sistemi operativi moderni, i processi non esistono in isolamento: spesso un processo ne genera altri, dando origine a una gerarchia di processi. Questo meccanismo permette di organizzare l'esecuzione dei programmi, gestire le risorse in modo ordinato e facilitare il controllo dei processi attivi.

Il concetto chiave è che un **processo può creare altri processi**, detti *processi figli*, dando origine a una struttura ad albero.

11.2 Creazione di processi figli

Il processo che genera altri processi viene chiamato **processo padre** (*parent process*), mentre i processi generati vengono detti **processi figli** (*child processes*).

La creazione di un processo figlio segue generalmente questi passaggi:

1. Il processo padre invoca una specifica chiamata di sistema per creare un nuovo processo.
2. Il sistema operativo alloca un nuovo PCB per il processo figlio.
3. Il contesto iniziale del processo figlio viene impostato, compreso lo stato iniziale, il program counter e lo stack.
4. Il nuovo processo entra nella *ready queue* e sarà eseguito secondo le regole di scheduling.

Ogni processo figlio eredita alcune caratteristiche dal processo padre, come:

- privilegi e identità dell'utente
- alcune risorse condivise (es. file aperti)
- ambiente di esecuzione di base

Tuttavia, ogni processo figlio possiede anche:

- un proprio PCB
- uno stack separato
- uno stato indipendente

11.3 Struttura ad albero dei processi

Quando i processi creano altri processi, si forma una struttura gerarchica nota come **albero dei processi** (*process tree*).

Caratteristiche principali dell'albero dei processi:

- Il **processo radice** è il primo processo del sistema o del sottoinsieme gestito dall'utente.
- Ogni nodo dell'albero rappresenta un processo.
- Ogni arco indica una relazione padre-figlio tra processi.
- I processi possono avere più figli, formando rami dell'albero.

Questa struttura consente al sistema operativo di:

- tracciare facilmente le relazioni tra processi
- gestire la terminazione dei processi figli quando il padre termina
- controllare l'allocazione delle risorse e la propagazione di segnali

11.4 Identificatori dei processi

Ogni processo nel sistema operativo riceve un **Process ID** (*PID*) univoco. Questo identificatore permette al sistema operativo di distinguere ogni processo dagli altri.

Il PID è essenziale per:

- gestire le comunicazioni tra processi
- inviare segnali a processi specifici
- monitorare le risorse utilizzate da ciascun processo

11.5 Il processo *init* in Linux

In sistemi Unix-like, come Linux, esiste un processo speciale chiamato **init** che rappresenta l'antenato di tutti i processi utente.

Caratteristiche principali di *init*:

- Ha PID 1.
- Viene avviato dal kernel durante il boot del sistema.
- Crea i primi processi utente, che a loro volta possono creare altri processi figli.
- Garantisce che ogni processo del sistema abbia un antenato comune.

In altre parole, *init* funge da radice dell'albero dei processi in Linux, assicurando una gerarchia ben definita e facilitando la gestione dei processi nel sistema operativo.

11.6 Osservazione conclusiva

La creazione dei processi e la struttura ad albero sono fondamentali per:

- organizzare i processi in modo gerarchico
- tracciare relazioni padre-figlio
- gestire efficacemente le risorse del sistema
- supportare la terminazione e il controllo dei processi

Questi concetti costituiscono la base per meccanismi avanzati come la propagazione di segnali, la gestione dei processi zombie e orfani, e l'implementazione di politiche di sicurezza e isolamento tra processi.

12 Allocazione delle Risorse ai Processi Figli

12.1 Introduzione

Quando un processo padre crea un processo figlio, il nuovo processo necessita di una serie di risorse per poter eseguire correttamente le proprie attività. La gestione e l'assegnazione di queste risorse sono fondamentali per garantire il corretto funzionamento del sistema e prevenire sovraccarichi.

Le principali risorse richieste da un processo figlio includono:

- tempo di CPU
- memoria
- file aperti
- dispositivi di input/output

12.2 Origine delle risorse

Le risorse necessarie a un processo figlio possono provenire da due fonti principali:

1. **Direttamente dal sistema operativo:** il kernel assegna nuove risorse al processo figlio, come memoria separata, nuovi descrittori di file o l'accesso alla CPU tramite lo scheduler.
2. **Dal pool di risorse del processo padre:** il processo padre può decidere di condividere parte delle proprie risorse con il figlio, come file aperti, memoria condivisa o periferiche già inizializzate.

Questa flessibilità consente al sistema operativo di ottimizzare l'utilizzo delle risorse e di supportare modelli di programmazione diversi.

12.3 Condivisione e divisione delle risorse

Il processo padre ha la possibilità di:

- **Condividere risorse:** il figlio e il padre utilizzano insieme alcune risorse, ad esempio un file aperto o una porzione di memoria condivisa. Questo facilita la comunicazione e la cooperazione tra processi.
- **Dividere risorse:** il padre può allocare al figlio una porzione limitata di risorse, come una quantità definita di memoria o un numero massimo di file aperti. In questo modo, si evita che il figlio consumi risorse in eccesso.

La scelta tra condivisione e divisione dipende dal tipo di applicazione, dal modello di sicurezza e dalle politiche di gestione delle risorse del sistema operativo.

12.4 Limitazione delle risorse del figlio

Imporre limiti alle risorse assegnate a un processo figlio è importante per:

- prevenire il sovraccarico del sistema
- garantire che il processo padre continui a funzionare correttamente
- evitare che un figlio monopolizzi CPU, memoria o dispositivi di I/O

Il sistema operativo può applicare diversi meccanismi di limitazione, come:

- quote di memoria
- limiti sul numero massimo di file aperti
- restrizioni sul tempo di CPU assegnato

Queste misure proteggono il sistema da comportamenti anomali o da errori nei programmi figli.

12.5 Osservazioni conclusive

La gestione delle risorse dei processi figli è un aspetto critico dei sistemi operativi. Una corretta allocazione permette di:

- garantire stabilità e sicurezza del sistema
- facilitare la cooperazione tra processi
- ottimizzare l'uso della CPU e della memoria
- prevenire condizioni di overload o deadlock

Pertanto, i sistemi operativi moderni integrano politiche sofisticate di allocazione e condivisione delle risorse per gestire in maniera equilibrata i processi figli e le risorse globali.

13 Comportamento dei Processi Padre e Figlio

13.1 Introduzione

Dopo la creazione di un processo figlio, il sistema operativo deve gestire il comportamento sia del processo padre sia del processo figlio. Esistono diverse opzioni di esecuzione e di gestione dello spazio di indirizzamento che determinano come i due processi interagiscono e utilizzano le risorse.

13.2 Opzioni di esecuzione

Riguardo all'esecuzione, un processo padre può comportarsi in due modi principali rispetto al processo figlio:

1. **Esecuzione concorrente:** il processo padre continua la propria esecuzione contemporaneamente al figlio. In questo caso, entrambi i processi possono essere eseguiti in parallelo, soggetti alle regole di scheduling della CPU. Questo approccio permette il massimo sfruttamento della CPU e facilita la concorrenza tra processi.

2. **Attesa del completamento del figlio:** il processo padre sospende temporaneamente la propria esecuzione fino a quando il processo figlio non termina. Questo comportamento è utile quando il padre deve utilizzare i risultati prodotti dal figlio o quando si vuole garantire una sequenza di esecuzione deterministica.

La scelta tra esecuzione concorrente e attesa dipende dalle esigenze dell'applicazione e dalle politiche del sistema operativo.

13.3 Opzioni di spazio di indirizzamento

Oltre all'esecuzione, il processo figlio deve essere configurato in termini di **spazio di indirizzamento**, ovvero come il processo figlio utilizza la memoria rispetto al padre. Le principali opzioni sono:

1. **Duplicazione del padre:** il figlio riceve una copia del programma e dei dati del processo padre. In questo modo, padre e figlio hanno spazi di indirizzamento separati ma inizialmente identici. Questa opzione è utile quando il figlio deve continuare a eseguire lo stesso programma del padre, ma con dati indipendenti.

2. **Caricamento di un nuovo programma:** il processo figlio può caricare un programma completamente diverso. In questo caso, lo spazio di indirizzamento del figlio viene inizializzato con il nuovo programma e i relativi dati. Questa opzione è tipica dei sistemi Unix quando si utilizza la combinazione `fork` + `exec`, permettendo al figlio di eseguire un programma distinto dal padre.

13.4 Osservazioni conclusive

La gestione del comportamento di padre e figlio è cruciale per:

- determinare la concorrenza tra processi
- garantire l'indipendenza o la condivisione dei dati
- supportare scenari in cui il padre dipende dai risultati del figlio
- ottimizzare l'utilizzo delle risorse e la reattività del sistema

Comprendere queste opzioni permette agli sviluppatori e ai sistemi operativi di controllare in modo preciso la sequenza di esecuzione e l'allocazione dello spazio di memoria nei processi generati.

14 Creazione dei Processi in UNIX

14.1 Introduzione

Nei sistemi operativi Unix e Unix-like, la creazione e gestione dei processi segue una logica specifica basata su due chiamate di sistema fondamentali: `fork()` e `exec()`. Queste consentono di generare nuovi processi, controllare il loro spazio di indirizzamento e determinare il comportamento di esecuzione tra padre e figlio.

14.2 La chiamata `fork()`

La funzione `fork()` è utilizzata per creare un nuovo processo:

- Il processo padre invoca `fork()`.
- Il sistema operativo crea un **processo figlio** duplicando lo spazio di indirizzamento del padre, comprese le sezioni di codice, dati, heap e stack.
- Il figlio riceve una copia indipendente delle risorse del padre, mentre entrambi condividono inizialmente solo ciò che è necessario per l'efficienza (es. pagine di memoria in scrittura su copia).

14.2.1 Valore di ritorno di `fork()`

La chiamata `fork()` restituisce valori diversi al padre e al figlio, permettendo di distinguere i due processi:

- Al **figlio** restituisce 0.
- Al **padre** restituisce il PID (Process ID) del figlio appena creato.

Questo meccanismo consente di scrivere codice condizionale per determinare quale parte del programma deve essere eseguita dal padre e quale dal figlio.

14.3 La chiamata `exec()`

La funzione `exec()` serve a sostituire lo spazio di indirizzamento di un processo con un nuovo programma:

- Il processo mantiene lo stesso PID.
- Il contenuto della memoria del processo, comprese le sezioni di codice, dati, heap e stack, viene sostituito con quello del nuovo programma.
- I registri, il program counter e lo stack vengono inizializzati secondo il nuovo programma.

L'uso di `exec()` è tipico quando un processo figlio, creato con `fork()`, deve eseguire un programma diverso dal padre.

14.4 Opzioni del processo padre

Dopo aver creato un figlio con `fork()`, il processo padre può scegliere tra due comportamenti principali:

1. **Continuare l'esecuzione concorrente:** il padre prosegue immediatamente la propria esecuzione, permettendo al figlio e al padre di eseguire in parallelo secondo le regole di scheduling della CPU.
2. **Chiamare `wait()`:** il padre sospende la propria esecuzione fino a quando il figlio termina. La chiamata `wait()` restituisce il PID del processo figlio terminato e permette al padre di recuperare lo stato di uscita del figlio.

14.5 Osservazioni conclusive

La combinazione `fork()` + `exec()` costituisce un meccanismo potente nei sistemi Unix per:

- creare nuovi processi in modo efficiente
- determinare se il padre e il figlio devono eseguire in parallelo o in sequenza
- caricare nuovi programmi nei processi figli senza perdere il PID
- controllare in maniera flessibile lo spazio di indirizzamento e le risorse condivise

Questi strumenti forniscono una base coerente per costruire applicazioni concorrenti, gestire pipeline di processi e implementare server multitasking in ambiente Unix.

15 Terminazione dei Processi

15.1 Introduzione

La terminazione dei processi è una fase critica del ciclo di vita di un processo nei sistemi operativi. Essa garantisce che le risorse allocate siano liberate correttamente e che il sistema rimanga stabile e coerente. La gestione della terminazione coinvolge sia il processo stesso sia il suo processo padre, che può monitorarne lo stato e, in alcuni casi, controllarne la fine.

15.2 Terminazione volontaria del processo

Un processo può terminare volontariamente invocando la chiamata di sistema `exit()`:

- La funzione `exit()` segnala al sistema operativo che il processo ha completato la propria esecuzione.
- Il sistema operativo aggiorna lo stato del processo a *Terminated*.
- Tutte le risorse allocate al processo — memoria, file aperti, buffer di I/O — vengono rilasciate e rese disponibili per altri processi.

15.3 Recupero dello stato del processo figlio

Il processo padre può ottenere informazioni sul completamento di un figlio tramite la chiamata di sistema `wait()`:

- `wait()` sospende il processo padre fino alla terminazione del figlio.
- Restituisce il PID del processo figlio terminato.
- Fornisce lo stato di uscita del figlio, utile per determinare se l'esecuzione è terminata con successo o con errori.

Questo meccanismo permette al padre di sincronizzarsi con il figlio e di gestire correttamente le risorse condivise.

15.4 Terminazione forzata da parte del processo padre

In alcuni casi, il processo padre può decidere di terminare uno o più figli, ad esempio quando:

- il figlio supera i limiti di risorse assegnate (CPU, memoria, file aperti)
- il compito del figlio non è più necessario
- il processo padre stesso termina e non è consentito che i figli continuino l'esecuzione autonomamente

Il sistema operativo fornisce strumenti per inviare segnali ai processi figli affinché terminino in modo controllato.

15.5 Terminazione a cascata

Il concetto di **cascading termination** si riferisce alla situazione in cui:

- la terminazione di un processo padre comporta la terminazione automatica di tutti i suoi processi figli.
- Questo meccanismo impedisce che processi figli rimangano attivi senza un padre, evitando processi orfani non gestiti.

La terminazione a cascata è essenziale per mantenere la coerenza della gerarchia dei processi e per garantire che tutte le risorse vengano correttamente liberate.

15.6 Osservazioni conclusive

La terminazione dei processi, sia volontaria sia forzata, è fondamentale per:

- liberare risorse allocate, evitando perdite di memoria o blocchi di I/O
- garantire la sincronizzazione tra padre e figli
- mantenere la stabilità e la sicurezza del sistema operativo
- evitare processi orfani o zombie

La gestione accurata della terminazione e della cascata di terminazioni permette ai sistemi operativi di assicurare un controllo completo sul ciclo di vita dei processi, ottimizzando l'uso delle risorse e prevenendo comportamenti anomali.

16 Terminazione dei Processi e Processi Zombie

16.1 Terminazione dei processi figli

Quando un processo figlio termina la propria esecuzione, il meccanismo tipico nei sistemi operativi Unix-like prevede:

1. Il figlio invoca la chiamata di sistema `exit(status)`, segnalando al kernel che ha completato la propria esecuzione e fornendo uno stato di uscita (`status`) che indica successo o tipo di errore.
2. Il processo padre può chiamare `wait(&status)` per recuperare lo stato di uscita del figlio.
3. La funzione `wait()` restituisce il PID del figlio terminato e permette al padre di gestire correttamente eventuali dati di ritorno o risorse condivise.

Questo processo assicura che il padre sia informato della terminazione del figlio e che le risorse allocate al processo figlio vengano liberate correttamente.

16.2 Processo zombie

Un **processo zombie** è un processo che ha terminato la propria esecuzione, ma la sua entry nella **process table** rimane ancora presente. Le caratteristiche principali di un processo zombie sono:

- Il processo ha eseguito `exit()` e tutte le risorse principali (memoria, file, buffer I/O) sono state rilasciate.
- La voce nella tabella dei processi persiste per permettere al processo padre di leggere lo stato di uscita.
- Il PID rimane occupato finché il padre non chiama `wait()`.

16.3 Rimozione del processo zombie

Quando il processo padre invoca `wait()`:

- Lo stato di uscita del figlio viene letto e restituito al padre.
- L'entry nella process table viene eliminata.
- Il PID del processo figlio viene rilasciato e può essere riutilizzato per nuovi processi.

16.4 Osservazioni conclusive

I processi zombie rappresentano uno stato transitorio e necessario per il corretto funzionamento della comunicazione tra padre e figlio:

- Permettono al padre di recuperare informazioni sul completamento del figlio.
- Non consumano memoria o risorse significative, ma occupano una voce nella tabella dei processi.
- Una gestione inefficace dei processi zombie può portare al **process table overflow**, impedendo la creazione di nuovi processi.

La comprensione del concetto di processo zombie è cruciale per amministratori e sviluppatori, poiché consente di prevenire problemi di gestione dei processi e garantire che il sistema operativo mantenga una struttura di processi coerente e stabile.

17 Processi Indipendenti vs Processi Cooperanti

17.1 Introduzione

Nei sistemi operativi, i processi possono essere classificati in base al grado di interazione e dipendenza tra di essi. Questa distinzione è importante per comprendere come i processi condividono dati, influenzano l'esecuzione reciproca e collaborano nella realizzazione di un compito più ampio.

17.2 Processi indipendenti

Un **processo indipendente** è un processo che non condivide dati con altri processi e la cui esecuzione non viene influenzata dagli altri processi. Caratteristiche principali:

- Non scambia informazioni con altri processi.
- Non può influenzare né essere influenzato dall'esecuzione di altri processi.
- È tipicamente più semplice da progettare e gestire, poiché non richiede meccanismi di sincronizzazione.

Esempi di processi indipendenti includono programmi batch o calcoli isolati che non dipendono da input esterni condivisi.

17.3 Processi cooperanti

Un **processo cooperante** è invece un processo che può interagire e condividere dati con altri processi. Caratteristiche principali:

- Condivide informazioni e risorse con altri processi.
- Può influenzare l'esecuzione degli altri processi e viceversa.
- Richiede meccanismi di sincronizzazione e comunicazione per evitare problemi come condizioni di race o incoerenza dei dati.

Esempi tipici includono processi che accedono a database condivisi, server-client, pipeline di elaborazione dati e sistemi multithreaded.

17.4 Perché la cooperazione è importante

La cooperazione tra processi è fondamentale per diversi motivi:

1. **Condivisione delle informazioni:** processi cooperanti permettono agli utenti o ad altre applicazioni di accedere a dati comuni, garantendo coerenza e aggiornamenti in tempo reale.

2. **Esecuzione più veloce:** la collaborazione tra processi o thread consente di suddividere un compito complesso in più attività parallele, riducendo i tempi di completamento.

3. **Progettazione modulare:** la separazione di un sistema in processi o thread distinti permette di sviluppare moduli indipendenti che cooperano tra loro, migliorando manutenibilità e scalabilità del software.

17.5 Osservazioni conclusive

La distinzione tra processi indipendenti e cooperanti è essenziale per:

- progettare sistemi operativi efficienti
- determinare quali meccanismi di comunicazione e sincronizzazione sono necessari
- bilanciare sicurezza, isolamento e collaborazione tra processi

Comprendere queste differenze consente di scegliere il modello più adatto alle esigenze di un'applicazione e di ottimizzare la gestione delle risorse del sistema.

18 Comunicazione tra Processi (IPC)

18.1 Introduzione

La **Interprocess Communication (IPC)** è l'insieme dei meccanismi mediante i quali i processi possono scambiarsi informazioni e coordinare le proprie attività. L'IPC è fondamentale nei sistemi operativi per consentire la cooperazione tra processi, specialmente quando questi condividono dati o risorse.

Esistono due principali modelli di IPC:

1. **Memoria condivisa (Shared Memory)**
2. **Scambio di messaggi (Message Passing)**

18.2 Modello a Memoria Condivisa

Nel modello della memoria condivisa, più processi accedono a una regione di memoria comune:

- La memoria condivisa è creata e gestita dal sistema operativo.
- I processi possono leggere e scrivere direttamente nella memoria condivisa.
- L'accesso deve essere coordinato tramite meccanismi di sincronizzazione (es. semafori, mutex) per evitare condizioni di race.

Vantaggi della memoria condivisa:

- Alta velocità di comunicazione, poiché non è necessario passare continuamente per il kernel.
- Efficienza nella gestione di grandi quantità di dati.

Ruolo del kernel: il kernel interviene principalmente per creare e configurare la regione di memoria condivisa, dopodiché i processi possono comunicare autonomamente.

18.3 Modello a Scambio di Messaggi

Nel modello di **message passing**, i processi comunicano inviando e ricevendo messaggi tramite il kernel:

- Ogni messaggio contiene dati e, talvolta, informazioni di controllo.
- Il kernel gestisce la coda dei messaggi, l'invio e la ricezione, assicurando correttezza e sincronizzazione.
- I messaggi possono essere bloccanti (blocking) o non bloccanti (non-blocking).

Vantaggi dello scambio di messaggi:

- Semplicità nella programmazione, soprattutto per piccole quantità di dati.
- Facilità di utilizzo in sistemi distribuiti, dove i processi possono risiedere su macchine diverse.
- Non richiede sincronizzazione esplicita nella memoria condivisa.

18.4 Confronto tra i due modelli

- **Velocità:** la memoria condivisa è generalmente più veloce, poiché l'accesso ai dati non passa costantemente per il kernel.
- **Semplicità:** il message passing è più semplice da implementare, soprattutto per piccole quantità di dati o sistemi distribuiti.
- **Scopo:** la memoria condivisa è più adatta a trasferimenti di grandi volumi di dati tra processi sulla stessa macchina, mentre il message passing è preferibile per sistemi distribuiti o comunicazioni puntuali.

18.5 Osservazioni conclusive

La scelta del modello di IPC dipende da:

- quantità di dati da trasferire
- complessità della sincronizzazione richiesta
- architettura del sistema (locale o distribuita)
- requisiti di efficienza e reattività

Entrambi i modelli sono strumenti essenziali per costruire applicazioni concorrenti e sistemi cooperativi, garantendo che i processi possano condividere informazioni in modo sicuro e coordinato.

19 Sistemi a Memoria Condivisa

19.1 Introduzione

Nei sistemi a memoria condivisa, una porzione dello spazio di indirizzamento di un processo può essere resa accessibile ad altri processi o thread per consentire la comunicazione diretta. Questo modello di IPC permette uno scambio di dati rapido e flessibile senza l'intervento continuo del kernel.

19.2 Attacco della regione condivisa

Per comunicare tramite memoria condivisa, i processi cooperanti devono:

- **Attaccare** (*attach*) la regione di memoria condivisa al proprio spazio di indirizzamento.
- Una volta attaccata, leggere e scrivere direttamente nella memoria condivisa come se fosse parte del proprio spazio di memoria.

19.3 Gestione della memoria condivisa

Dopo l'allocazione della memoria condivisa:

- La gestione della memoria diventa responsabilità dei processi stessi.
- Il sistema operativo interviene principalmente nella creazione, nella configurazione iniziale e nella rimozione della regione condivisa.
- I processi determinano tipo, dimensioni e posizione dei dati condivisi.

19.4 Controllo della concorrenza

Poiché più processi o thread possono accedere contemporaneamente alla memoria condivisa, è essenziale garantire la consistenza dei dati:

- I processi devono evitare di scrivere simultaneamente nella stessa locazione di memoria.
- L'accesso concorrente viene tipicamente controllato tramite meccanismi di sincronizzazione come **semafori**, **mutex** o **barriere**.

19.5 Osservazioni conclusive

I sistemi a memoria condivisa offrono vantaggi notevoli:

- Comunicazione ad alta velocità tra processi sulla stessa macchina.
- Controllo diretto dei dati e della struttura della memoria condivisa.
- Flessibilità nel design di sistemi cooperativi e applicazioni multithreaded.

Tuttavia, richiedono attenzione particolare nella sincronizzazione per prevenire **race conditions** e garantire la coerenza dei dati. La corretta gestione della memoria condivisa è quindi un aspetto critico nella progettazione di sistemi concorrenti.

20 Problema del Produttore-Consumatore

20.1 Introduzione

Il **problema del produttore-consumatore** è un esempio classico di processi o thread cooperanti in cui la sincronizzazione è fondamentale. Esso illustra come due tipi di processi — il produttore e il consumatore — possano condividere una risorsa comune, tipicamente un buffer, senza corrompere i dati né bloccare il sistema.

20.2 Descrizione del problema

- **Produttore:** genera elementi di dati e li inserisce nel buffer condiviso.
- **Consumatore:** preleva e utilizza gli elementi dal buffer.

Per garantire la correttezza:

- Il produttore non deve aggiungere elementi se il buffer è pieno.
- Il consumatore non deve leggere elementi se il buffer è vuoto.

La sincronizzazione è quindi necessaria per coordinare le operazioni dei due processi e prevenire condizioni di *race* o accessi invalidi.

20.3 Approcci principali

Esistono due approcci principali per implementare il problema del produttore-consumatore:

20.3.1 Buffer illimitato (Unbounded Buffer)

- Il buffer non ha limiti pratici di dimensione.
- Il produttore può generare nuovi dati senza restrizioni.
- Il consumatore potrebbe dover attendere che nuovi elementi vengano prodotti se il buffer è temporaneamente vuoto.
- Non è necessario bloccare il produttore, ma occorre gestire eventuali attese del consumatore.

20.3.2 Buffer limitato (Bounded Buffer)

- Il buffer ha una dimensione fissa, limitata.
- Se il buffer è pieno, il produttore deve attendere finché il consumatore non preleva un elemento.
- Se il buffer è vuoto, il consumatore deve attendere finché il produttore non aggiunge un elemento.
- Richiede meccanismi di sincronizzazione espliciti, come semafori o mutex, per coordinare correttamente produttore e consumatore.

20.4 Osservazioni conclusive

Il problema del produttore-consumatore illustra concetti chiave di cooperazione tra processi:

- Necessità di sincronizzazione per evitare conflitti sull'accesso a risorse condivise.
- Differenze tra gestione di buffer illimitati e limitati.
- Applicabilità pratica a scenari reali come code di messaggi, pipeline di dati, e sistemi multithreaded.

Comprendere questo problema fornisce una base per affrontare altri problemi di sincronizzazione e coordinamento nei sistemi concorrenti.

21 Scambio di Messaggi nella Comunicazione tra Processi (IPC)

21.1 Introduzione

Il **message passing** è un modello di comunicazione tra processi che consente lo scambio di informazioni senza utilizzare memoria condivisa. Questo modello è particolarmente importante nei sistemi distribuiti, dove i processi possono risiedere su macchine diverse, e non è possibile accedere a uno spazio di memoria comune.

21.2 Operazioni di base

Le due operazioni fondamentali del message passing sono:

- `send(message)`: permette a un processo di inviare un messaggio a un altro processo.
- `receive(message)`: permette a un processo di ricevere un messaggio da un altro processo.

Il kernel o il sistema di comunicazione garantisce che i messaggi siano consegnati correttamente tra i processi mittenti e destinatari.

21.3 Tipi di messaggi

I messaggi possono essere classificati in base alla lunghezza:

- **Lunghezza fissa:** ogni messaggio ha una dimensione predeterminata. Questo semplifica la gestione da parte del sistema operativo ma può rendere più complessa la programmazione, ad esempio per adattarsi a dati di dimensioni variabili.
- **Lunghezza variabile:** i messaggi possono avere dimensioni diverse. Questo aumenta la flessibilità e rende più semplice la programmazione, ma il sistema deve gestire una maggiore complessità nella memorizzazione e trasmissione dei messaggi.

21.4 Metodi di comunicazione

Il message passing può essere ulteriormente classificato in base al metodo di trasmissione:

1. **Diretta vs Indiretta:**
   - Diretta: il mittente invia il messaggio direttamente al destinatario specificato.
   - Indiretta: il messaggio passa attraverso un *mailbox* o coda, che può essere condivisa tra più processi.
2. **Sincrona vs Asincrona:**
   - Sincrona (bloccante): il mittente attende finché il destinatario non riceve il messaggio.
   - Asincrona (non bloccante): il mittente continua l'esecuzione senza attendere che il destinatario legga il messaggio.
3. **Buffering automatico vs esplicito:**
   - Automatico: il sistema mantiene un buffer per memorizzare temporaneamente i messaggi fino alla ricezione.
   - Esplicito: i processi devono gestire direttamente il buffering dei messaggi.

21.5 Osservazioni conclusive

Il message passing offre un metodo sicuro e flessibile di comunicazione tra processi, con vantaggi principali:

- Consente la cooperazione tra processi senza condividere memoria.
- È essenziale per sistemi distribuiti.
- Offre diversi modelli di comunicazione adattabili alle esigenze dell'applicazione, bilanciando semplicità, velocità e complessità di implementazione.

Comprendere le operazioni fondamentali, i tipi di messaggi e i metodi di comunicazione è cruciale per progettare sistemi concorrenti e distribuire processi in modo efficace.

22 Naming e Comunicazione Diretta/Indiretta

22.1 Introduzione

Nei sistemi a message passing, l'identificazione dei processi destinatari e mittenti è fondamentale per garantire la corretta trasmissione dei messaggi. Esistono due modalità principali di comunicazione basate sul naming dei processi: **comunicazione diretta** e **comunicazione indiretta** tramite mailbox.

22.2 Comunicazione Diretta

22.2.1 Primitive

Le primitive di comunicazione diretta consentono di inviare e ricevere messaggi specificando direttamente il processo destinatario o mittente:

- `send(P, message)`: invia un messaggio al processo identificato da P.
- `receive(Q, message)`: riceve un messaggio dal processo identificato da Q.

22.2.2 Caratteristiche

La comunicazione diretta possiede le seguenti caratteristiche:

1. Il collegamento (*link*) tra i processi comunicanti viene creato automaticamente dal sistema operativo.
2. Ogni link connette esattamente due processi: un mittente e un ricevente.
3. Esiste un solo link tra ciascuna coppia di processi, evitando ambiguità nella trasmissione dei messaggi.

22.2.3 Varianti

- **Simmetrica:** sia il mittente sia il ricevente conoscono e nominano l'altro processo.
- **Asimmetrica:** solo il mittente conosce e nomina il ricevente. La primitive di ricezione assume la forma `receive(id, message)`.

22.2.4 Limitazioni

La comunicazione diretta presenta un limite significativo di modularità: cambiare il nome di un processo richiede di aggiornare tutte le definizioni e le chiamate in cui tale processo è nominato. Ciò può rendere complessa la manutenzione di sistemi estesi.

22.3 Comunicazione Indiretta (Mailbox)

22.3.1 Primitive

Nella comunicazione indiretta, i messaggi non sono inviati direttamente a un processo specifico, ma a un **mailbox** condiviso:

- `send(A, message)`: invia un messaggio alla mailbox identificata da A.
- `receive(A, message)`: riceve un messaggio dalla mailbox identificata da A.

22.3.2 Caratteristiche

- Il collegamento tra processi esiste solo se condividono la stessa mailbox.
- Una singola mailbox può essere condivisa tra più processi.
- È possibile creare diverse mailbox tra gli stessi processi, aumentando la modularità.

22.3.3 Problemi e soluzioni

Quando più riceventi condividono una mailbox (ad esempio P_1, P_2, P_3), si pone la domanda: *chi riceve il messaggio?*.

Le possibili soluzioni includono:

1. Limitare la mailbox a due processi (mittente e ricevente).
2. Consentire una sola operazione `receive()` alla volta.
3. Lasciare che il sistema selezioni automaticamente il ricevente tramite un algoritmo di selezione.

22.4 Osservazioni conclusive

La scelta tra comunicazione diretta e indiretta influisce su:

- modularità del sistema
- complessità della gestione dei nomi dei processi
- flessibilità nel collegamento tra più processi
- possibilità di implementare algoritmi di selezione dei destinatari

Comprendere questi meccanismi è essenziale per progettare sistemi concorrenti robusti e scalabili, dove più processi cooperano tramite messaggi in modo sicuro ed efficiente.

23 Sincronizzazione nella Comunicazione tra Processi

23.1 Introduzione

La sincronizzazione è un aspetto fondamentale nella comunicazione tra processi tramite message passing. Garantire che i messaggi vengano inviati e ricevuti correttamente richiede la gestione del comportamento bloccante o non bloccante di operazioni di invio e ricezione.

23.2 Tipi di invio (Send)

- **Synchronous send (invio sincrono):** il processo mittente rimane bloccato fino a quando il messaggio non è stato consegnato al destinatario. Questo garantisce che il destinatario riceva effettivamente il messaggio prima che il mittente continui l'esecuzione.
- **Asynchronous send (invio asincrono):** il mittente invia il messaggio e continua immediatamente l'esecuzione senza attendere la conferma di consegna. Questo approccio riduce il tempo di attesa del mittente, ma può introdurre complessità nella gestione della consegna dei messaggi.

23.3 Tipi di ricezione (Receive)

- **Synchronous receive (ricezione sincrona):** il processo ricevente rimane bloccato fino a quando un messaggio non arriva nella coda o nella mailbox corrispondente.
- **Asynchronous receive (ricezione asincrona):** il processo ricevente preleva un messaggio se presente; se non ci sono messaggi, riceve un valore nullo (`null`) e continua l'esecuzione.

23.4 Comportamento combinato

Quando sia il mittente sia il ricevente utilizzano operazioni bloccanti, si verifica un **rendezvous** tra i due processi:

- Il mittente attende che il ricevente sia pronto.
- Il ricevente attende che il mittente invii il messaggio.
- La comunicazione avviene solo quando entrambi i processi sono sincronizzati.

23.5 Esempio: Problema del Produttore-Consumatore

- Il **produttore** utilizza una `send()` bloccante per assicurarsi che il messaggio sia consegnato al consumatore.
- Il **consumatore** utilizza una `receive()` bloccante per attendere l'arrivo di un nuovo elemento prodotto.
- Questo schema garantisce che il produttore non produca più dati di quanti il buffer o il consumatore possano gestire, evitando overflow o letture da buffer vuoto.

23.6 Osservazioni conclusive

La sincronizzazione nei sistemi a message passing è essenziale per:

- garantire la correttezza della comunicazione tra processi
- prevenire condizioni di race e accessi inconsistenti ai dati
- coordinare produttore e consumatore o processi cooperanti
- scegliere il giusto compromesso tra blocco del mittente e efficienza di esecuzione

Una gestione attenta delle modalità sincrone e asincrone consente di costruire sistemi concorrenti affidabili e performanti.

24 Buffering nella Comunicazione tra Processi

24.1 Introduzione

Il **buffering** è il meccanismo mediante il quale i messaggi vengono temporaneamente memorizzati in code prima di essere ricevuti dai processi destinatari. La gestione della capacità del buffer determina se l'invio e la ricezione dei messaggi saranno operazioni bloccanti o non bloccanti.

24.2 Tipi di buffer

24.2.1 Capacità zero (Zero Capacity)

- Non è possibile accumulare messaggi in attesa.
- Il mittente deve attendere finché il ricevente non è pronto a ricevere il messaggio.
- Questo corrisponde a comunicazione completamente sincrona tra mittente e ricevente.

24.2.2 Capacità limitata (Bounded Capacity)

- La coda ha una dimensione massima fissa n messaggi.
- Se la coda non è piena, il mittente può inviare il messaggio e continuare l'esecuzione.
- Se la coda è piena, il mittente deve attendere che uno spazio diventi disponibile.
- Questo approccio richiede la gestione della concorrenza per evitare condizioni di overflow e garantire la correttezza della comunicazione.

24.2.3 Capacità illimitata (Unbounded Capacity)

- La coda può crescere indefinitamente senza limiti pratici.
- Il mittente non deve mai attendere per inviare un messaggio.
- Questo approccio semplifica la logica del mittente, ma richiede attenzione nella gestione della memoria per evitare saturazioni.

24.3 Idea chiave

La capacità della coda di messaggi determina se la comunicazione sarà:

- **Bloccante per il mittente:** il mittente deve attendere finché il messaggio non può essere consegnato.
- **Non bloccante per il mittente:** il mittente invia il messaggio e prosegue immediatamente, senza attese.

24.4 Osservazioni conclusive

La scelta della capacità del buffer ha impatti significativi su:

- sincronizzazione tra mittente e ricevente
- efficienza e reattività dei processi
- complessità della gestione della coda e del kernel

Un corretto dimensionamento e gestione del buffer è essenziale per garantire comunicazioni affidabili e prevenire blocchi o perdite di messaggi in sistemi concorrenti.

25 Socket e Comunicazione di Rete

25.1 Introduzione

Le **sockets** sono strumenti fondamentali per la comunicazione tra processi in reti di computer. Esse permettono lo scambio di dati tra processi che risiedono su macchine diverse, seguendo modelli come client–server o peer-to-peer.

25.2 Definizione di Socket

- Una **socket** è un endpoint di un canale di comunicazione tra due processi.
- Ogni socket è identificata in modo univoco da una combinazione di:
  - **Indirizzo IP** della macchina
  - **Numero di porta** associato al processo
- Le sockets sono utilizzate principalmente nelle architetture **client–server**, dove un server attende richieste e un client le invia.

25.3 Porte di comunicazione

- Il **server** rimane in ascolto su una porta specifica in attesa di richieste dai client.
- Le porte con numero minore di 1024 (*well-known ports*) sono riservate a servizi standard come HTTP (80), FTP (21), SSH (22).
- Le porte dei client sono generalmente assegnate in modo casuale e superiore a 1024.

25.4 Esempio di comunicazione client–server

- Client: (146.86.5.20 : 1625)
- Server: (161.25.19.8 : 80)
- Ogni connessione tra client e server è rappresentata da una **coppia di socket** unica, che identifica in modo univoco il canale di comunicazione tra le due estremità.

25.5 Idea chiave

La creazione di coppie di socket distinte per ogni connessione garantisce:

- comunicazioni simultanee tra più client e server
- affidabilità nello scambio di dati
- isolamento dei flussi di comunicazione tra diverse sessioni

25.6 Osservazioni conclusive

Le sockets rappresentano un modello versatile e scalabile per la comunicazione di rete:

- consentono la connessione tra processi su macchine differenti
- supportano architetture client–server e peer-to-peer
- permettono la gestione simultanea di più connessioni tramite identificatori unici (IP + porta)
- forniscono la base per protocolli di rete standard come TCP e UDP

Comprendere il concetto di socket è essenziale per sviluppare applicazioni di rete robuste e scalabili, garantendo che ogni processo possa comunicare in modo sicuro e affidabile.

26 Remote Procedure Call (RPC)

26.1 Introduzione

Il **Remote Procedure Call (RPC)** è un meccanismo che permette a un processo di invocare procedure su un sistema remoto come se fossero procedure locali. Questo modello astrae le complessità della comunicazione di rete, consentendo ai programmatori di utilizzare chiamate strutturate anziché gestire direttamente i dati grezzi.

26.2 Meccanismo di funzionamento

Il funzionamento di RPC si basa sui seguenti passaggi:

1. Il processo chiamante invia una **richiesta** al **RPC daemon** sul sistema remoto.
2. Il messaggio di richiesta include:
   - l'identificatore della funzione da eseguire
   - i parametri necessari per la funzione
3. Il sistema remoto esegue la funzione richiesta e restituisce il **risultato** al processo chiamante.

Il processo chiamante percepisce l'esecuzione come una normale chiamata locale, senza preoccuparsi dei dettagli di trasmissione, serializzazione dei parametri o gestione della rete.

26.3 Porte di servizio

- Ogni servizio RPC è associato a un **numero di porta** specifico su cui il daemon ascolta le richieste.
- Esempio: la porta 3027 può essere riservata a un servizio che fornisce l'elenco degli utenti attivi su un sistema remoto.

26.4 Idea chiave

Il concetto centrale di RPC è che esso permette a un processo di:

- eseguire procedure su sistemi remoti come se fossero locali
- astrarre la complessità della comunicazione di rete
- lavorare con messaggi strutturati invece di dati grezzi

26.5 Osservazioni conclusive

RPC facilita lo sviluppo di applicazioni distribuite affidabili:

- semplifica la programmazione client–server su più macchine
- consente l'invocazione di funzioni remote senza gestire manualmente la serializzazione dei dati
- integra la gestione delle porte e dei servizi remoti in modo trasparente

Comprendere RPC è essenziale per progettare sistemi distribuiti efficienti e modulari, dove i processi possono cooperare attraverso procedure remote come se fossero locali.

27 Thread

27.1 Introduzione

Un **thread** è l'unità di base di utilizzo della CPU all'interno di un processo. Rappresenta un flusso di controllo sequenziale con stato proprio e permette ai processi di eseguire più attività contemporaneamente.

27.2 Struttura di un thread

Ogni thread include:

- **ID del thread:** identificatore unico all'interno del processo.
- **Program Counter (PC):** indica l'istruzione corrente da eseguire.
- **Registri della CPU:** contengono lo stato temporaneo necessario per l'esecuzione.
- **Stack:** memorizza i dati locali e gli indirizzi di ritorno per le chiamate di funzione.

27.3 Condivisione tra thread dello stesso processo

I thread appartenenti allo stesso processo condividono alcune risorse:

- **Sezione di codice:** tutte le istruzioni del programma.
- **Sezione dati:** variabili globali e strutture dati comuni.
- **File aperti:** descrittori di file e socket condivisi.
- **Segnali e altre risorse di processo.**

27.4 Tipi di processi in base ai thread

- **Processo a thread singolo (single-threaded):** chiamato anche *heavyweight process*, può eseguire una sola attività alla volta.
- **Processo multithreaded:** può eseguire più attività concorrenti tramite thread separati, migliorando l'efficienza e la reattività.

27.5 Idea chiave

Il multithreading migliora le prestazioni dei processi permettendo:

- esecuzione simultanea di compiti differenti
- migliore utilizzo della CPU
- maggiore reattività delle applicazioni

27.6 Motivazioni per il multithreading

Le applicazioni moderne sono quasi sempre multithreaded per:

- suddividere compiti complessi in thread specializzati
- permettere l'esecuzione concorrente di operazioni di I/O e computazione
- aumentare la modularità e la scalabilità del software

27.7 Esempi di applicazioni multithreaded

- Applicazioni di elaborazione immagini: un thread per ciascuna miniatura.
- Browser web: thread dedicato alla visualizzazione dell'interfaccia e thread separato per il download dei contenuti.
- Word processor: thread per l'interfaccia utente, thread per l'input e thread per controlli in background.

27.8 Vantaggi nei sistemi multicore

- I thread possono essere eseguiti in parallelo su core differenti, aumentando le prestazioni complessive.
- Ottimizza l'utilizzo delle risorse hardware e riduce i tempi di risposta delle applicazioni.

27.9 Osservazioni conclusive

Il concetto di thread è essenziale per comprendere la concorrenza nei sistemi operativi moderni:

- permette processi più leggeri e reattivi rispetto ai processi tradizionali "heavyweight"
- facilita l'implementazione di applicazioni complesse e parallele
- consente di sfruttare appieno le architetture multicore

28 Processi vs Thread in un Server Web

28.1 Introduzione

Nei server web, l'architettura scelta per gestire le richieste dei client ha un impatto diretto sulle prestazioni e sull'efficienza. Si possono distinguere due approcci principali: server a singolo processo e server multithreaded.

28.2 Server a singolo processo

- Per ogni richiesta di client, il server crea un nuovo **processo** separato.
- La creazione del processo è **lenta e dispendiosa in termini di risorse**, poiché include l'allocazione di memoria, stack, registri, PCB e altre strutture del sistema operativo.
- Questo approccio può limitare la scalabilità e la reattività del server quando le richieste aumentano.

28.3 Server multithreaded

- Un singolo **processo** gestisce tutte le richieste dei client.
- Per ciascuna richiesta, il server crea un **thread** separato.
- I thread condividono codice e risorse del processo principale, riducendo l'overhead rispetto alla creazione di nuovi processi.
- Questo approccio è più **veloce, leggero e scalabile**, permettendo di gestire molteplici client in parallelo senza consumare risorse eccessive.

28.4 Idea chiave

L'uso dei **thread** nei server web consente:

- concorrenza efficiente tra più richieste
- riduzione dell'overhead rispetto alla creazione di processi
- migliore utilizzo delle risorse di sistema

28.5 Osservazioni conclusive

Confrontare processi e thread evidenzia:

- I thread forniscono una forma più leggera di concorrenza rispetto ai processi "heavyweight".
- Le applicazioni server moderne preferiscono il multithreading per la gestione di più client simultanei.
- La scelta tra processi e thread dipende dalla complessità delle operazioni, dal numero di client e dalle risorse hardware disponibili.

Questa analisi è cruciale per progettare server web efficienti, scalabili e reattivi.

29 Vantaggi del Multithreading

29.1 Introduzione

Il **multithreading** offre numerosi benefici per le applicazioni moderne e per la gestione dei processi nei sistemi operativi. Grazie alla possibilità di eseguire più thread all'interno dello stesso processo, è possibile migliorare la reattività, l'efficienza e la scalabilità delle applicazioni.

29.2 Principali vantaggi

1. **Reattività (Responsiveness):** Un programma multithreaded può continuare a funzionare anche se una parte è temporaneamente bloccata (ad esempio in attesa di I/O). Ciò garantisce che l'interfaccia utente o altre funzionalità critiche rimangano operative.

2. **Condivisione delle risorse (Resource Sharing):** I thread condividono automaticamente la memoria e le risorse del processo padre, come sezioni di codice, variabili globali e file aperti. Questo riduce la duplicazione delle risorse e facilita la cooperazione tra thread.

3. **Economia (Economy):** La creazione di un thread e il cambio di contesto tra thread sono operazioni più veloci e meno costose rispetto alla creazione e gestione di processi completi ("heavyweight"). Ciò permette di gestire numerosi thread con minore overhead di sistema.

4. **Scalabilità (Scalability):** I thread possono essere eseguiti in parallelo su più core della CPU, migliorando le prestazioni complessive e sfruttando appieno le architetture multicore.

29.3 Osservazioni conclusive

Il multithreading rappresenta un paradigma chiave per applicazioni moderne e sistemi operativi:

- Migliora l'efficienza nell'uso della CPU.
- Consente la gestione simultanea di più attività concorrenti.
- Riduce i costi di creazione e gestione rispetto ai processi tradizionali.
- Facilita lo sviluppo di applicazioni reattive, modulari e scalabili.

Comprendere questi benefici è essenziale per progettare software concorrente efficiente e ottimizzato per sistemi multicore.

30 Sistemi Multicore e Multithreading

30.1 Introduzione

I sistemi moderni utilizzano CPU multicore per migliorare le prestazioni dei processi e delle applicazioni. Il multithreading permette di sfruttare appieno questi core per ottenere vero parallelismo, aumentando l'efficienza e la reattività delle applicazioni.

30.2 Caratteristiche dei sistemi multicore

- Le CPU moderne contengono più **core**, ovvero unità di elaborazione indipendenti.
- Ogni core è percepito dal sistema operativo come un **processore separato**, capace di eseguire thread autonomamente.
- I sistemi possono essere:
  - **Multicore:** più core su un singolo chip.
  - **Multiprocessore:** più chip fisici, ciascuno con uno o più core.

30.3 Multithreading nei sistemi multicore

- Il **multithreading** permette di distribuire i thread di un processo su più core.
- Questo consente **vero parallelismo**: thread diversi eseguono istruzioni simultaneamente su core distinti.
- In un sistema a singolo core, i thread sembrano eseguire **concorrenza**, ma in realtà vengono intercalati (*interleaving*) dal sistema operativo, non eseguiti in parallelo.

30.4 Idea chiave

Il multithreading combinato con sistemi multicore consente:

- sfruttamento ottimale delle risorse hardware
- riduzione dei tempi di esecuzione complessivi
- miglioramento della reattività e throughput delle applicazioni

30.5 Osservazioni conclusive

Comprendere il comportamento dei thread su sistemi multicore è essenziale per:

- progettare applicazioni efficienti e scalabili
- distinguere tra concorrenza (single-core) e parallelismo reale (multicore)
- ottimizzare l'uso dei core per processi multithreaded

L'adozione di CPU multicore e il corretto utilizzo del multithreading rappresentano strumenti fondamentali per lo sviluppo di sistemi moderni ad alte prestazioni.

31 Concorrenza vs. Parallelismo

31.1 Introduzione

Nei sistemi multicore, è importante distinguere tra **concorrenza** e **parallelismo**, due concetti spesso correlati ma distinti:

- **Concorrenza:** più attività avanzano insieme nel tempo, ma non necessariamente eseguono istruzioni simultaneamente. È tipica dei sistemi single-core con interleaving dei thread.
- **Parallelismo:** più attività eseguono istruzioni **allo stesso tempo** su core differenti. È possibile solo in sistemi multicore o multiprocessore.

31.2 Possibili scenari

- Un sistema può essere **concurrent** senza essere **parallel**. Esempio: thread intercalati su un singolo core avanzano insieme, ma non sono eseguiti simultaneamente.
- Nei sistemi multicore, la concorrenza può evolversi in vero parallelismo quando i thread vengono distribuiti su core distinti.

31.3 Tipi di parallelismo

Esistono due principali forme di parallelismo nei sistemi multithreaded:

31.3.1 Parallelismo sui dati (Data Parallelism)

- I dati vengono suddivisi tra più core.
- La stessa operazione viene eseguita simultaneamente su ciascun sottoinsieme di dati.
- Esempio: calcolo della somma di un array diviso in segmenti, ogni core elabora un segmento.

31.3.2 Parallelismo sui task (Task Parallelism)

- Diverse attività o thread vengono distribuite tra i core.
- Ogni thread esegue un'operazione differente.
- I thread possono operare sugli stessi dati o su dati diversi.
- Esempio: in un'applicazione web, un thread gestisce I/O, un altro thread calcola statistiche, un terzo thread aggiorna l'interfaccia utente.

31.4 Idea chiave

- **Data parallelism:** stessa attività, dati diversi.
- **Task parallelism:** attività diverse, dati possibili comuni o distinti.

31.5 Osservazioni conclusive

Comprendere la distinzione tra concorrenza e parallelismo è essenziale per:

- progettare applicazioni efficienti su sistemi multicore
- scegliere il modello di parallelismo più adatto al problema
- ottimizzare la distribuzione dei thread e dei dati tra i core

Questa distinzione guida lo sviluppo di software concorrente e parallelo, consentendo prestazioni elevate e sfruttamento ottimale delle risorse hardware.

32 Thread a Livello Utente vs Thread a Livello Kernel

32.1 Introduzione

Nei sistemi operativi moderni, i thread possono essere gestiti a due livelli:

- **User-level threads (ULT):** gestiti da librerie a livello utente senza supporto diretto del kernel.
- **Kernel-level threads (KLT):** gestiti direttamente dal sistema operativo.

La relazione tra thread a livello utente e thread a livello kernel definisce il **modello di esecuzione** e le possibilità di parallelismo e concorrenza.

32.2 Modelli comuni di mapping

1. **Many-to-One**
2. **One-to-One**
3. **Many-to-Many**

32.3 Modello Many-to-One

- Più thread a livello utente condividono un singolo thread a livello kernel.
- La gestione è affidata a una libreria utente, garantendo un'elevata efficienza nell'allocazione dei thread.

32.3.1 Limitazioni

- Se un thread esegue una chiamata bloccante, **tutti i thread del processo sono bloccati**.
- Solo un thread alla volta può accedere al kernel.
- Non è possibile eseguire thread in parallelo su sistemi multicore.

32.3.2 Risultato

- Modello semplice ma inefficiente.
- Oggi è raramente utilizzato a causa delle limitazioni sul parallelismo e la concorrenza.

32.4 Modello One-to-One

- Ogni thread a livello utente corrisponde a un thread a livello kernel.
- Supporta **vero parallelismo** su sistemi multiprocessore o multicore.
- Se un thread si blocca, gli altri possono continuare l'esecuzione.

32.4.1 Svantaggi

- Creare un thread a livello kernel per ciascun thread utente comporta un **overhead maggiore**.
- I sistemi spesso limitano il numero di thread per ridurre il costo in termini di prestazioni.

32.4.2 Idea chiave

- Migliore concorrenza rispetto al modello many-to-one.
- Più costoso in termini di risorse rispetto ai thread utente.

32.5 Modello Many-to-Many

- Molti thread utente vengono mappati su un numero uguale o minore di thread kernel.
- Il numero di thread kernel può dipendere dall'applicazione o dall'hardware (ad esempio, numero di core).
- Permette l'esecuzione parallela dei thread utente su sistemi multicore.
- Se un thread si blocca, il kernel può schedulare un altro thread kernel disponibile.

32.5.1 Vantaggi

- Uso flessibile ed efficiente delle risorse di sistema.

32.5.2 Svantaggi

- Complesso da implementare.
- Con i moderni sistemi multicore, l'implementazione many-to-many è meno necessaria.
- La maggior parte dei sistemi operativi moderni preferisce il modello one-to-one per semplicità e efficienza.

32.6 Idea chiave

- Il modello many-to-many bilancia flessibilità ed efficienza.
- Tuttavia, i sistemi moderni favoriscono la semplicità e l'affidabilità del mapping one-to-one.

32.7 Osservazioni conclusive

Conoscere i modelli di mapping dei thread è fondamentale per:

- progettare applicazioni concorrenti e parallele efficienti
- comprendere il compromesso tra overhead, concorrenza e parallelismo
- scegliere la strategia più adatta in base al sistema operativo e all'architettura hardware

33 Modello a Due Livelli (Two-Level Model)

33.1 Introduzione

Il **Two-Level Model** è una variante avanzata del modello *many-to-many* dei thread. Combina la flessibilità di eseguire molti thread utente su un numero limitato di thread kernel con la possibilità di legare alcuni thread utente a thread kernel specifici, ottenendo così maggiore controllo sulle operazioni critiche.

33.2 Funzionamento del modello

- Molti thread utente vengono mappati su un numero uguale o minore di thread kernel disponibili.
- Alcuni thread utente possono essere **bound** (legati) a un singolo thread kernel per garantire l'esecuzione indipendente di operazioni critiche o bloccanti.
- Gli altri thread utente **unbound** continuano a essere multiplexati in modo efficiente tra i thread kernel disponibili.

33.3 Vantaggi principali

- Combina la **flessibilità** del modello many-to-many con il **controllo** del modello one-to-one.
- I thread **bound** possono eseguire operazioni critiche dal punto di vista temporale o chiamate bloccanti in modo indipendente, senza influire sugli altri thread.
- I thread **unbound** mantengono un'esecuzione efficiente grazie al multiplexing su thread kernel condivisi.

33.4 Idea chiave

Il Two-Level Model permette di ottenere un compromesso ottimale tra:

- flessibilità nella gestione dei thread utente
- efficienza nell'utilizzo dei thread kernel
- controllo su thread che richiedono tempi di risposta critici o operazioni bloccanti

33.5 Osservazioni conclusive

Questo modello è particolarmente utile in applicazioni complesse e sistemi moderni multicore, dove alcune attività richiedono priorità o isolamento:

- Fornisce un bilanciamento tra prestazioni e affidabilità.
- Riduce l'overhead rispetto al modello one-to-one puro.
- Permette di gestire thread critici e non critici in modo ottimizzato.

34 Librerie per Thread

34.1 Introduzione

Le **librerie per thread** forniscono un **API** che consente la creazione, gestione e sincronizzazione dei thread all'interno di un processo. Esse astraggono i dettagli implementativi e permettono agli sviluppatori di lavorare con thread in modo semplice e coerente.

34.2 Implementazione delle librerie

Le librerie per thread possono essere implementate in due modalità principali:

- **User space:**
  - La gestione dei thread avviene completamente nello spazio utente, senza supporto diretto del kernel.
  - Operazioni come creazione, cancellazione e cambio di contesto sono veloci.
  - Limiti: non è possibile sfruttare il parallelismo reale sui sistemi multicore; se un thread si blocca, l'intero processo può essere bloccato.
- **Kernel space:**
  - La gestione dei thread è supportata dal sistema operativo.
  - Consente vero parallelismo su sistemi multicore.
  - Operazioni come cambio di contesto e sincronizzazione richiedono l'intervento del kernel, quindi più costose rispetto allo spazio utente.

34.3 Principali librerie per thread

- **POSIX Pthreads:** disponibile sia a livello utente che kernel su UNIX/Linux.
- **Win32 Threads:** a livello kernel su sistemi Windows.
- **Java Threads:** dipendono dalla JVM e dal sistema operativo sottostante (implementazione tramite Pthreads o Win32 Threads).

34.4 Esempio di utilizzo

Consideriamo il calcolo della somma degli interi da 0 a N in un thread separato:

- Creare un thread dedicato al calcolo della somma.
- Il thread esegue il ciclo e accumula il risultato in una variabile condivisa `sum`.
- Al termine, il thread termina e il risultato può essere utilizzato dal thread principale o da altri thread.

34.5 Osservazioni conclusive

Le librerie per thread sono fondamentali per:

- semplificare la programmazione concorrente
- astrarre la gestione dei thread e le differenze tra OS
- implementare applicazioni multithreaded efficienti e portabili

La scelta tra librerie a livello utente o kernel dipende dalle esigenze di parallelismo, performance e complessità del progetto.

35 Pthreads

35.1 Introduzione

Pthreads (POSIX Threads) è lo standard POSIX per la creazione e la sincronizzazione dei thread nei sistemi UNIX-like. Esso definisce il comportamento delle API per thread, ma non fornisce una specifica implementazione: Linux e macOS forniscono implementazioni concrete conformi allo standard.

35.2 Concetti chiave

- **Creazione dei thread:** ogni programma può creare uno o più thread per eseguire attività parallele.
- **Sincronizzazione dei thread:** meccanismi come `pthread_join()` e `pthread_exit()` permettono di coordinare la terminazione e la raccolta dei risultati dei thread.

35.3 Esempio pratico

Supponiamo di voler calcolare la somma degli interi da 0 a N utilizzando due thread:

- **Thread principale (Parent Thread):** rappresentato dalla funzione `main()`.
- **Thread figlio (Child Thread):** rappresentato dalla funzione `runner()` che esegue il calcolo.

35.3.1 Strategia Fork-Join

1. Il thread principale crea il thread figlio utilizzando `pthread_create()`.
2. Il thread principale aspetta la terminazione del thread figlio con `pthread_join()`.
3. Il thread figlio esegue il calcolo della somma e termina con `pthread_exit()`.
4. Il thread principale, dopo la terminazione del figlio, stampa il risultato condiviso `sum`.

35.4 Note sull'utilizzo multicore

- Su sistemi multicore, è comune creare più thread per sfruttare il parallelismo reale.
- La sincronizzazione di tutti i thread può essere gestita facilmente con un ciclo `for` che chiama `pthread_join()` su ciascun thread.
- Questo garantisce che tutti i thread abbiano completato l'esecuzione prima che il thread principale continui o termini.

35.5 Osservazioni conclusive

L'uso di Pthreads permette di:

- creare applicazioni multithreaded portabili su sistemi POSIX
- sincronizzare thread in modo semplice e sicuro
- sfruttare le architetture multicore per eseguire operazioni in parallelo
- implementare schemi Fork-Join per gestire attività parallele con raccolta dei risultati

36 Thread su Windows

36.1 Introduzione

I thread su Windows seguono un modello simile a Pthreads, ma utilizzano un **API proprietario** fornito dal sistema operativo. Questo consente la creazione, gestione e sincronizzazione dei thread in ambienti Windows.

36.2 Creazione dei thread

- È necessario includere la libreria principale: `#include <windows.h>`.
- I dati condivisi tra thread (ad esempio la variabile `Sum`) devono essere dichiarati globali.
- La funzione eseguita dal thread è definita con il tipo `LPVOID` come parametro:
  - Esempio: `DWORD WINAPI Summation(LPVOID param)`
- La creazione del thread avviene tramite `CreateThread()`, che richiede i seguenti parametri principali:
  - attributi di sicurezza (`SECURITY_ATTRIBUTES`)
  - dimensione dello stack
  - flag iniziale di start
  - puntatore alla funzione da eseguire

36.3 Sincronizzazione dei thread

- `WaitForSingleObject(handle, timeout)`: attende la terminazione di un singolo thread.
- `WaitForMultipleObjects(count, handleArray, waitAll, timeout)`: attende la terminazione di più thread.
  - `count`: numero di oggetti thread
  - `handleArray`: array dei thread da attendere
  - `waitAll`: flag che indica se aspettare tutti i thread
  - `timeout`: tempo massimo di attesa

36.4 Idea chiave

I thread su Windows:

- seguono un modello concettuale simile a Pthreads
- utilizzano però funzioni API e tipi di dato specifici del sistema Windows
- permettono gestione, sincronizzazione e parallelismo tra thread in modo controllato

36.5 Osservazioni conclusive

La comprensione dei thread su Windows è fondamentale per:

- sviluppare applicazioni multithreaded efficienti in ambiente Windows
- gestire correttamente dati condivisi e sincronizzazione tra thread
- sfruttare il parallelismo su sistemi multicore Windows

Questa conoscenza consente di realizzare applicazioni reattive e scalabili, simili a quelle implementate con Pthreads su sistemi UNIX/Linux.

37 Thread in Java

37.1 Introduzione

I **thread in Java** rappresentano il paradigma centrale per l'esecuzione concorrente dei programmi Java. Sono disponibili su tutte le piattaforme basate su JVM (Windows, Linux, macOS). Ogni programma Java possiede almeno un thread, il **thread principale**.

37.2 Modalità di creazione dei thread

Esistono due principali metodi per creare thread in Java:

1. **Estendere la classe `Thread`** e sovrascrivere il metodo `run()`.
2. **Implementare l'interfaccia `Runnable`** e definire il metodo `run()` (approccio più comune e flessibile).

37.3 Passaggi per la creazione di un thread

1. Creare un oggetto `Thread`.
2. Passare un'istanza di `Runnable` (nel caso dell'implementazione dell'interfaccia).
3. Chiamare il metodo `start()`, che:
    - inizializza il nuovo thread all'interno della JVM
    - invoca automaticamente il metodo `run()` del thread

37.4 Nota importante

- Il metodo `run()` non deve mai essere chiamato direttamente.
- Chiamare `run()` direttamente eseguirebbe il codice nel thread chiamante corrente, senza creare un nuovo thread.
- Per avviare un thread effettivo, è obbligatorio utilizzare `start()`.

37.5 Osservazioni conclusive

I thread Java consentono:

- esecuzione concorrente di compiti all'interno di un programma
- portabilità su qualsiasi sistema con JVM
- flessibilità nella progettazione di applicazioni multithreaded

Comprendere la differenza tra `start()` e `run()` è fondamentale per evitare errori di esecuzione e garantire corretto parallelismo.

38 Modello Fork-Join

38.1 Introduzione

Il **modello Fork-Join** è una strategia di programmazione multithreaded utilizzata per eseguire compiti paralleli. In questo modello, un thread principale (*parent*) crea uno o più thread figli (*child*) per suddividere il lavoro e successivamente attende la loro terminazione per raccogliere e combinare i risultati.

38.2 Funzionamento del Fork-Join

1. Il **thread parent** crea (fork) uno o più **thread figli**.
2. I thread figli eseguono compiti paralleli indipendenti.
3. Il thread parent chiama **join** su ciascun thread figlio per attendere la loro terminazione.
4. Dopo il join, il parent raccoglie e combina i risultati dei thread figli per completare il compito complessivo.

38.3 Idea chiave

- I compiti paralleli vengono **divisi** tra i thread (fork) e successivamente **sincronizzati** (join) per ottenere il risultato finale.

38.4 Uso di `fork()` e `exec()` in programmi multithreaded

- In un contesto multithreaded:
  - `fork()` può duplicare **tutti i thread** del processo o solo il thread chiamante.
  - `exec()` sostituisce l'intero processo, rimuovendo tutti i thread esistenti.
- Considerazioni pratiche:
  - Se `exec()` segue immediatamente `fork()`, solo il thread chiamante viene duplicato.
  - Se `exec()` è ritardato, tutti i thread del processo parent vengono duplicati.

38.5 Idea chiave aggiuntiva

- La duplicazione dei thread dopo `fork()` dipende dal momento in cui viene invocata la chiamata `exec()`.

38.6 Osservazioni conclusive

Il modello Fork-Join è essenziale per:

- sfruttare il parallelismo nelle applicazioni multithreaded
- gestire in modo controllato la creazione e terminazione dei thread
- comprendere il comportamento di `fork()` e `exec()` in contesti con più thread

Questa strategia permette di realizzare programmi concorrenti e paralleli efficienti, bilanciando la creazione dei thread e la raccolta dei risultati.

39 Gestione dei Segnali

39.1 Introduzione

Nei sistemi UNIX, i **segnali** sono meccanismi utilizzati per notificare ai processi eventi specifici. Essi permettono la comunicazione asincrona tra sistema operativo e processi, oppure tra processi stessi.

39.2 Tipologie di segnali

- **Segnali asincroni:** generati da eventi esterni al processo, come l'interruzione da tastiera (`Ctrl+C`) o un evento hardware.
- **Segnali sincroni:** generati da eventi interni al processo, come errori aritmetici (divisione per zero) o violazioni di memoria.

39.3 Processo di gestione dei segnali

Il flusso tipico di gestione di un segnale è:

1. Si verifica un evento → viene generato un segnale.
2. Il segnale viene inviato a un processo specifico.
3. Il processo riceve il segnale e lo gestisce secondo la logica definita.

39.4 Tipi di gestori (handlers)

- **Gestore predefinito (default handler):** fornito dal kernel, esegue azioni standard (ad esempio terminare il processo).
- **Gestore definito dall'utente (user-defined handler):** sostituisce il comportamento predefinito con una funzione personalizzata che gestisce il segnale.

39.5 Opzioni di gestione

- **Ignorare il segnale:** il processo continua l'esecuzione come se il segnale non fosse avvenuto.
- **Terminare il processo:** il segnale provoca la terminazione controllata del processo.

39.6 Idea chiave

I segnali consentono al sistema operativo e ai processi di comunicare **asincronamente** eventi importanti, fornendo meccanismi di notifica immediata e flessibile.

39.7 Osservazioni conclusive

La gestione dei segnali è essenziale per:

- implementare comportamenti reattivi nei processi
- gestire eventi critici o errori runtime
- fornire strumenti di comunicazione tra processi senza la necessità di polling continuo

40 Gestione dei Segnali in Sistemi Multithreaded

40.1 Introduzione

Nei sistemi multithreaded, la gestione dei segnali richiede ulteriori considerazioni rispetto ai processi single-thread, poiché bisogna decidere **quale thread** deve gestire il segnale ricevuto.

40.2 Comportamento dei segnali

- **Processi single-thread:** i segnali vengono inviati direttamente al processo e gestiti dal thread unico.
- **Processi multithread:** è necessario stabilire quale thread riceverà e gestirà il segnale.

40.3 Opzioni per l'invio dei segnali

1. Inviare il segnale al **thread correlato** all'evento.
2. Inviare il segnale a **tutti i thread** del processo.
3. Inviare il segnale a **thread specifici**.
4. Designare un **thread dedicato** per la gestione di tutti i segnali.

40.4 Gestione in UNIX

- La funzione `kill(pid_t pid, int signal)` invia un segnale a un processo specificato dal PID.
- I thread possono definire quali segnali accettare o bloccare mediante maschere di segnale (`sigmask`).

40.5 Gestione in Windows

- Windows non dispone di segnali nativi.
- È possibile emulare segnali usando **APC (Asynchronous Procedure Calls)**.
- Un thread può specificare una funzione che gestirà automaticamente un evento specifico, replicando la funzionalità dei segnali.

40.6 Idea chiave

In ambienti multithreaded, la gestione dei segnali richiede decisioni sulla destinazione del segnale, garantendo che:

- il segnale raggiunga il thread corretto
- i thread continuino a operare senza interferenze indesiderate
- la programmazione concorrente rimanga sicura ed efficiente

40.7 Osservazioni conclusive

Capire la gestione dei segnali in sistemi multithreaded è fondamentale per:

- evitare comportamenti imprevisti e blocchi
- sincronizzare correttamente eventi asincroni tra thread
- implementare applicazioni concorrenti robuste su UNIX e Windows

41 Cancellazione dei Thread

41.1 Introduzione

La **cancellazione dei thread** consente di terminare un thread prima che abbia completato il proprio compito. Questo meccanismo è utile in scenari in cui, ad esempio, più thread cercano un risultato e, una volta trovato da uno di essi, gli altri thread possono essere terminati per risparmiare risorse.

41.2 Tipi di cancellazione

1. **Cancellazione asincrona (Asynchronous Cancellation):** Il thread target viene terminato immediatamente senza attendere un punto specifico nel codice.
2. **Cancellazione differita (Deferred Cancellation):** Il thread target verifica periodicamente se deve terminare, eseguendo una terminazione ordinata e sicura.

41.3 Problematiche principali

- Un thread cancellato può detenere risorse o modificare dati condivisi.
- Il sistema operativo recupera la maggior parte delle risorse dopo una cancellazione asincrona, ma non tutte; alcune risorse o strutture dati condivise potrebbero rimanere in uno stato inconsistente.

41.4 Cancellazione differita

- Garantisce una terminazione più sicura e ordinata.
- La cancellazione viene effettuata solo quando il thread raggiunge un **punto di cancellazione** predefinito.
- Nei sistemi POSIX, la funzione `pthread_testcancel()` può essere utilizzata per implementare un punto di cancellazione manuale.

41.5 Idea chiave

- La cancellazione differita è preferibile per evitare perdite di risorse o inconsistenze nei dati condivisi.
- La cancellazione asincrona può essere rapida, ma richiede attenzione per prevenire effetti collaterali indesiderati.

41.6 Osservazioni conclusive

Gestire correttamente la cancellazione dei thread è essenziale per:

- garantire integrità dei dati condivisi
- evitare perdite di risorse o deadlock
- implementare applicazioni multithreaded efficienti e sicure

42 Dati Specifici del Thread (Thread-Specific Data, TLS)

42.1 Introduzione

Nei sistemi multithreaded, i thread condividono generalmente la memoria del processo, comprese le sezioni di codice, dati globali e file aperti. Tuttavia, in alcune situazioni, ogni thread potrebbe aver bisogno di avere **copie private** di determinati dati. Questi dati privati sono chiamati **Thread-Specific Data (TLS)**.

42.2 Differenza rispetto alle variabili locali

- **Variabili locali:** visibili solo all'interno della funzione in cui sono dichiarate; ogni chiamata alla funzione ha la propria copia della variabile.
- **Thread-Specific Data (TLS):** visibili a tutte le funzioni eseguite dallo stesso thread; ogni thread ha la propria copia indipendente dagli altri thread.

42.3 Supporto dei TLS

- La maggior parte delle librerie per thread (ad esempio Pthreads) e dei compilatori moderni fornisce meccanismi per creare e gestire TLS.
- Tipicamente, ogni TLS viene identificato tramite un **key** associata al thread, che permette di leggere e scrivere dati privati in modo sicuro.

42.4 Idea chiave

- TLS permette ai thread di mantenere stati o dati privati senza interferire con gli altri thread.
- Fornisce un meccanismo sicuro per dati persistenti all'interno dello stesso thread, garantendo l'isolamento necessario alla concorrenza.

42.5 Osservazioni conclusive

I TLS sono fondamentali per:

- evitare conflitti tra thread durante l'accesso a variabili condivise
- implementare strutture dati private persistenti in applicazioni multithreaded
- semplificare la programmazione concorrente mantenendo la sicurezza dei dati

43 Comunicazione tra Thread a Livello Kernel e Utente

43.1 Introduzione

Nei modelli **many-to-many** e **two-level**, la comunicazione tra thread a livello kernel e thread a livello utente è essenziale per:

- regolare dinamicamente il numero di thread kernel disponibili
- ottimizzare le prestazioni del sistema
- coordinare correttamente l'esecuzione dei thread utente e kernel

43.2 Lightweight Process (LWP)

- Un **Lightweight Process (LWP)** agisce come un **processore virtuale**.
- I thread a livello utente vengono schedulati sui LWPs.
- Ogni LWP corrisponde a un **thread kernel**.
- Se un thread kernel si blocca, anche il corrispondente LWP e i thread utente ad esso associati si bloccano.

43.3 Scheduler Activations

- Il kernel fornisce LWPs all'applicazione.
- L'applicazione schedula i thread utente sui LWPs disponibili.
- Il kernel notifica eventi tramite **upcalls**, che vengono gestiti dalla libreria dei thread a livello utente.

43.4 Nota importante

- È necessario avere un numero sufficiente di LWPs rispetto ai thread kernel bloccati.
- Questo evita ritardi nell'esecuzione dei thread utente e garantisce che il sistema rimanga reattivo.

43.5 Idea chiave

La comunicazione kernel–utente permette:

- una gestione efficiente dei thread utente su più thread kernel
- adattamento dinamico della quantità di thread kernel disponibili
- sincronizzazione di eventi tra kernel e libreria thread utente

43.6 Osservazioni conclusive

Comprendere LWPs e le scheduler activations è fondamentale per:

- implementare modelli many-to-many e two-level efficienti
- garantire la reattività dei thread utente anche in presenza di blocchi kernel
- ottimizzare il parallelismo e la concorrenza nelle applicazioni multithreaded avanzate